The Biological Learning Rules I

Quan Wen May 18

The Credit Assignment Problem

The Credit Assignment Problem

When a network produces an error, which synapses (among potentially billions) are responsible?

Spatial Credit

Which neurons and layers contributed to the output error?

Temporal Credit

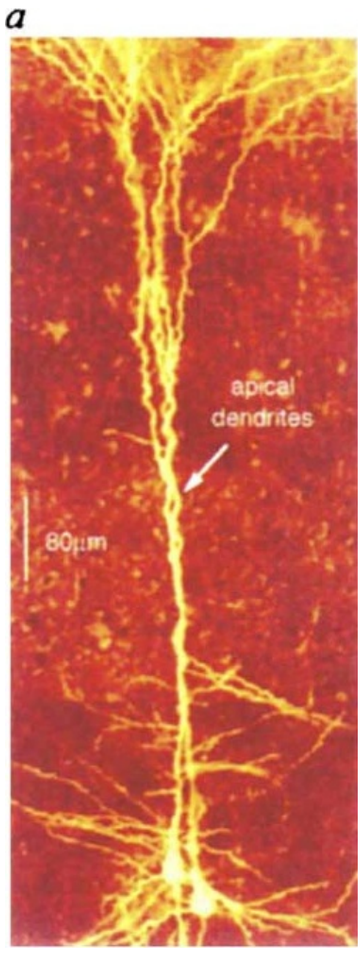
Which past activations (across time steps) are relevant?

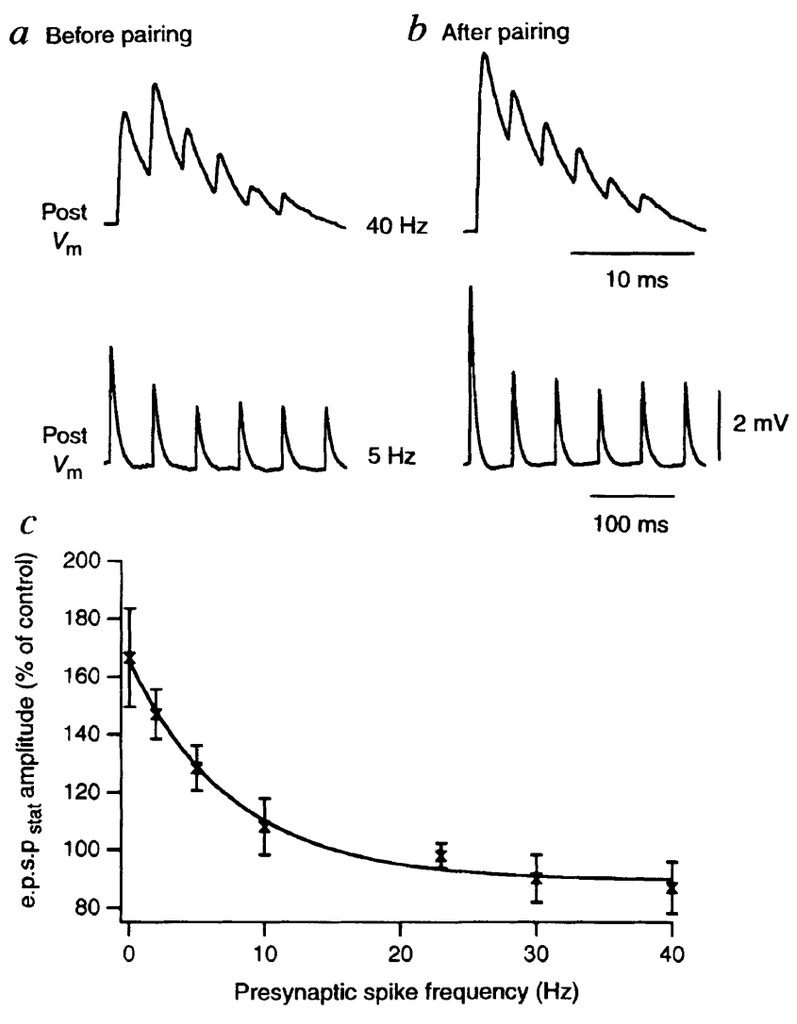
Structural Credit

How should each synapse's weight change to reduce error?

Synaptic plasticity (phenomenon)

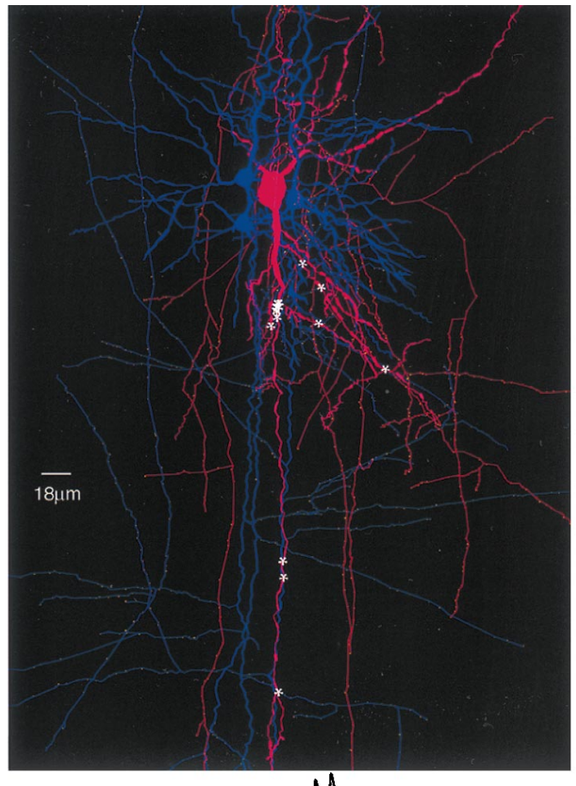
Short-term synaptic plasticity



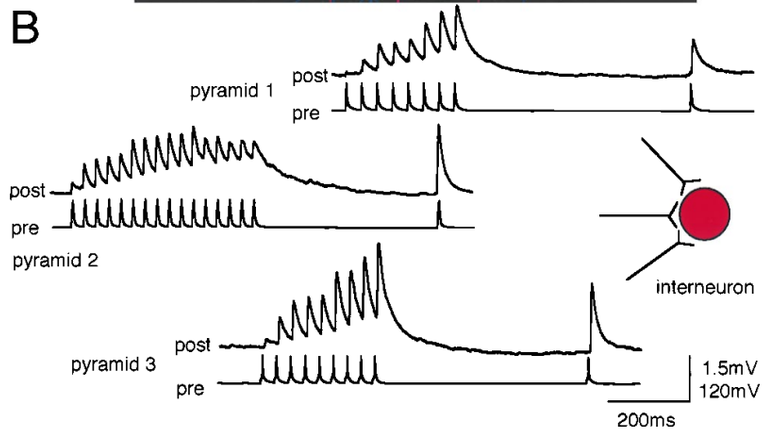


Depression

A



B



Facilitation

A Phenomenological Approach to Dynamic Synaptic Transmission

4 Key Synaptic Parameters

Absolute strength (A)
Probability of release (U)
Depression time constant (τ_d)
Facilitation time constant (τ_f)

2 Synaptic Variables

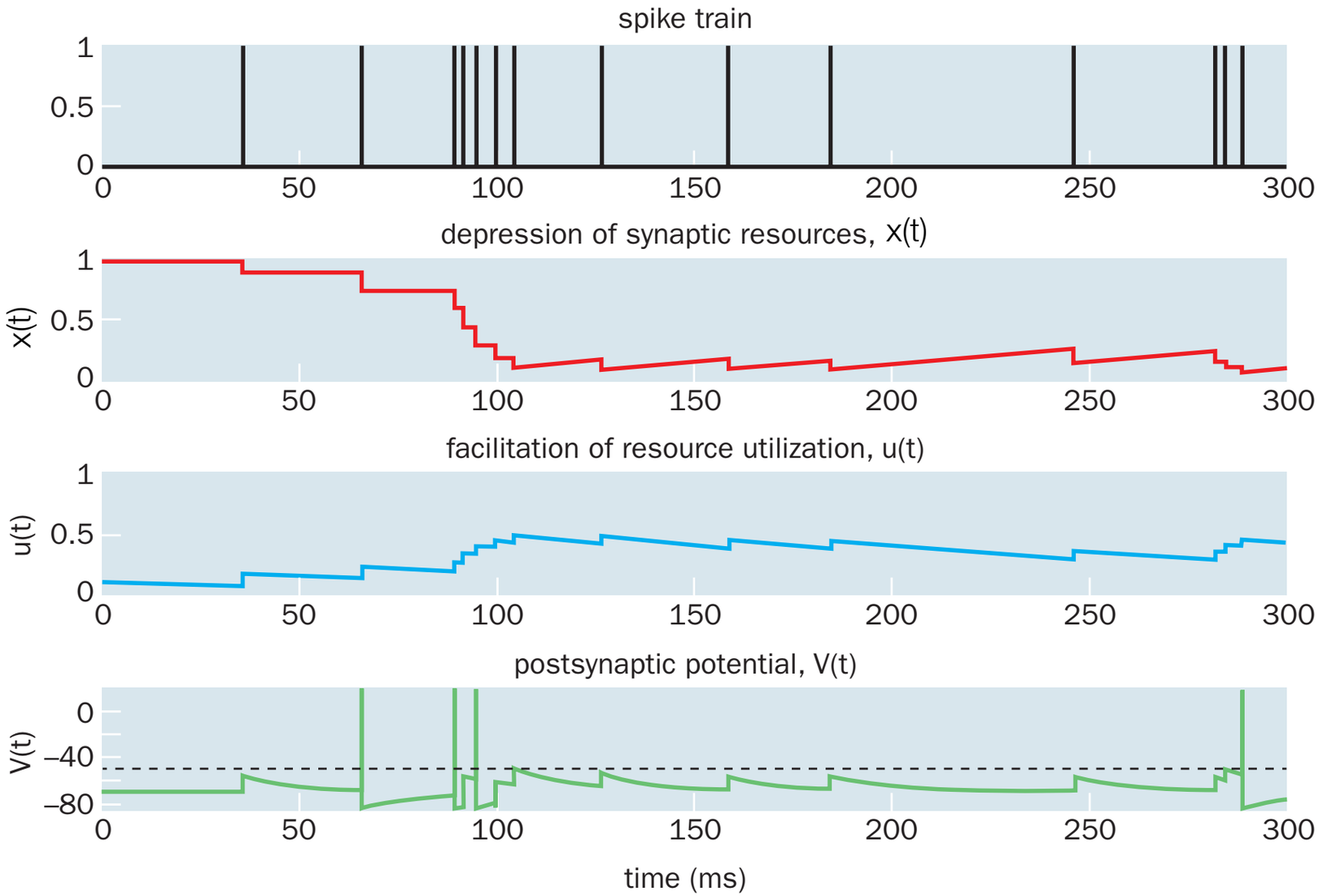
Resources available (x)
Release probability (u)

$$\frac{du}{dt} = -\frac{u}{\tau_f} + U(1-u)\delta(t - t_{spike})$$

$$\frac{dx}{dt} = -\frac{1-x}{\tau_d} - ux\delta(t - t_{spike})$$

$$V(t) = Aux$$

A Phenomenological Approach to Dynamic Synaptic Transmission



$$\frac{du}{dt} = -\frac{u}{\tau_f} + U(1-u)\delta(t - t_{spike})$$

$$\frac{dx}{dt} = -\frac{1-x}{\tau_d} - ux\delta(t - t_{spike})$$

$$V(t) = Aux$$

Long-term synaptic plasticity

Hebbian Learning

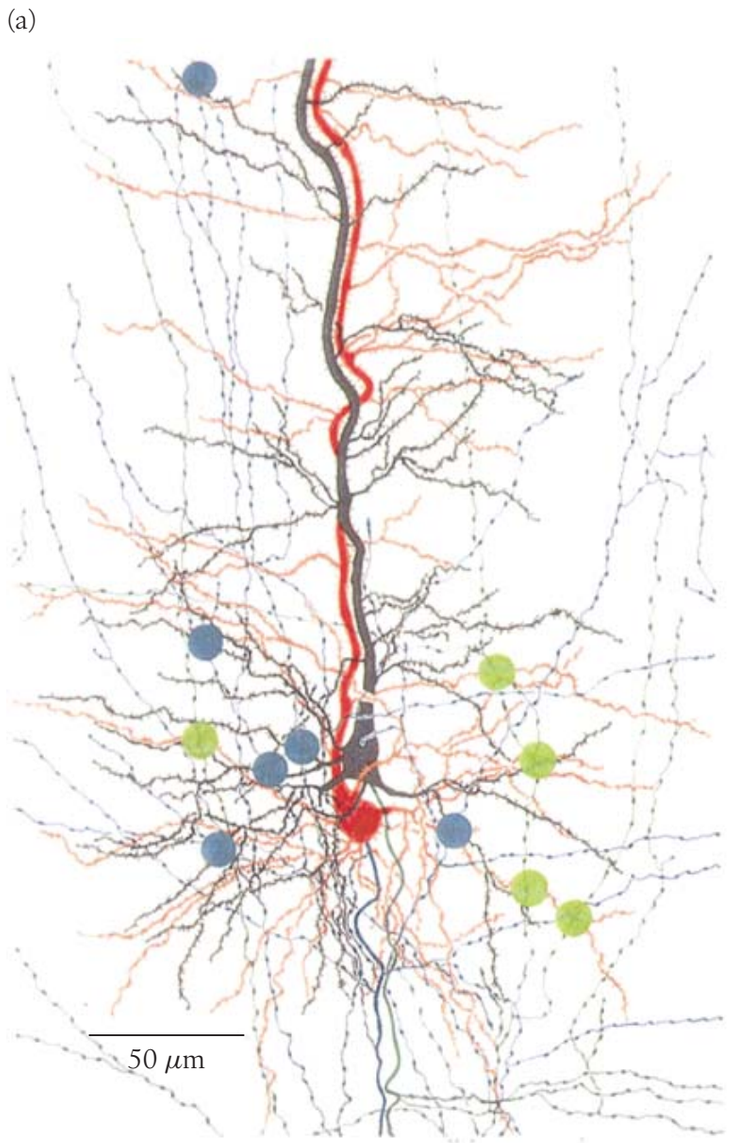
> "*Neurons that fire together, wire together*"
> — *Paraphrase of Hebb (1949)*

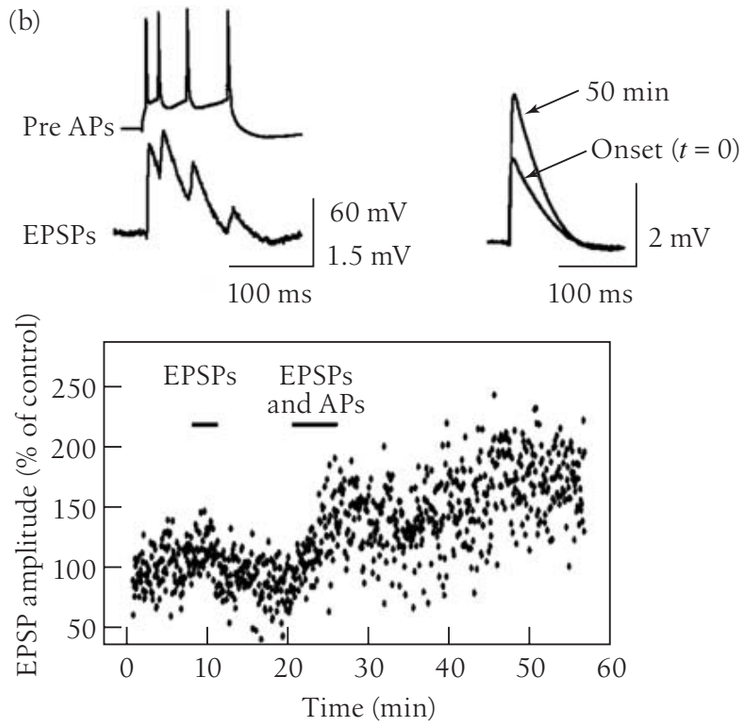
$$\Delta w_{ij} = \eta x_i x_j$$

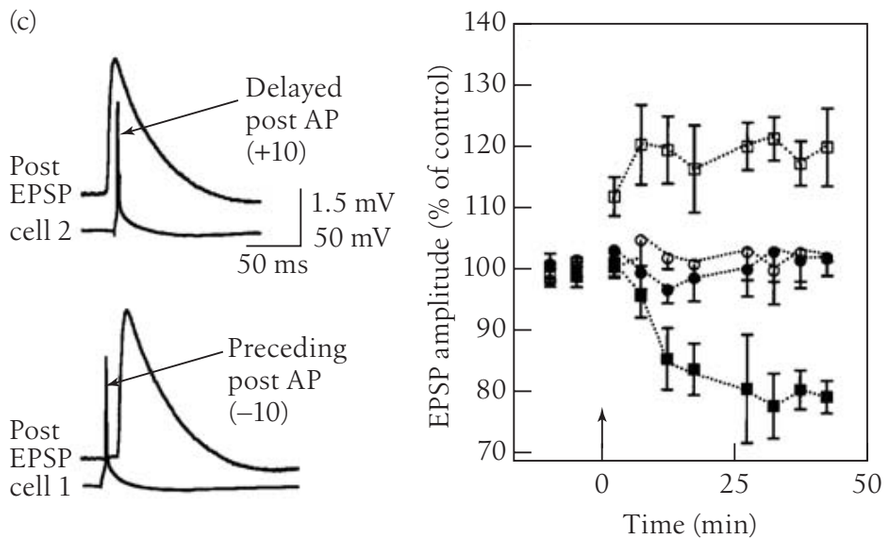
Δw_{ij}	Change in synaptic weight
η	Learning rate
$x_i,\ x_j$	Pre- and post-synaptic activity

- Local rule — only uses information available at the synapse
- Unsupervised — no external error signal required
- Unstable — weights grow without bound (needs normalization)

Spike-timing-dependent plasticity

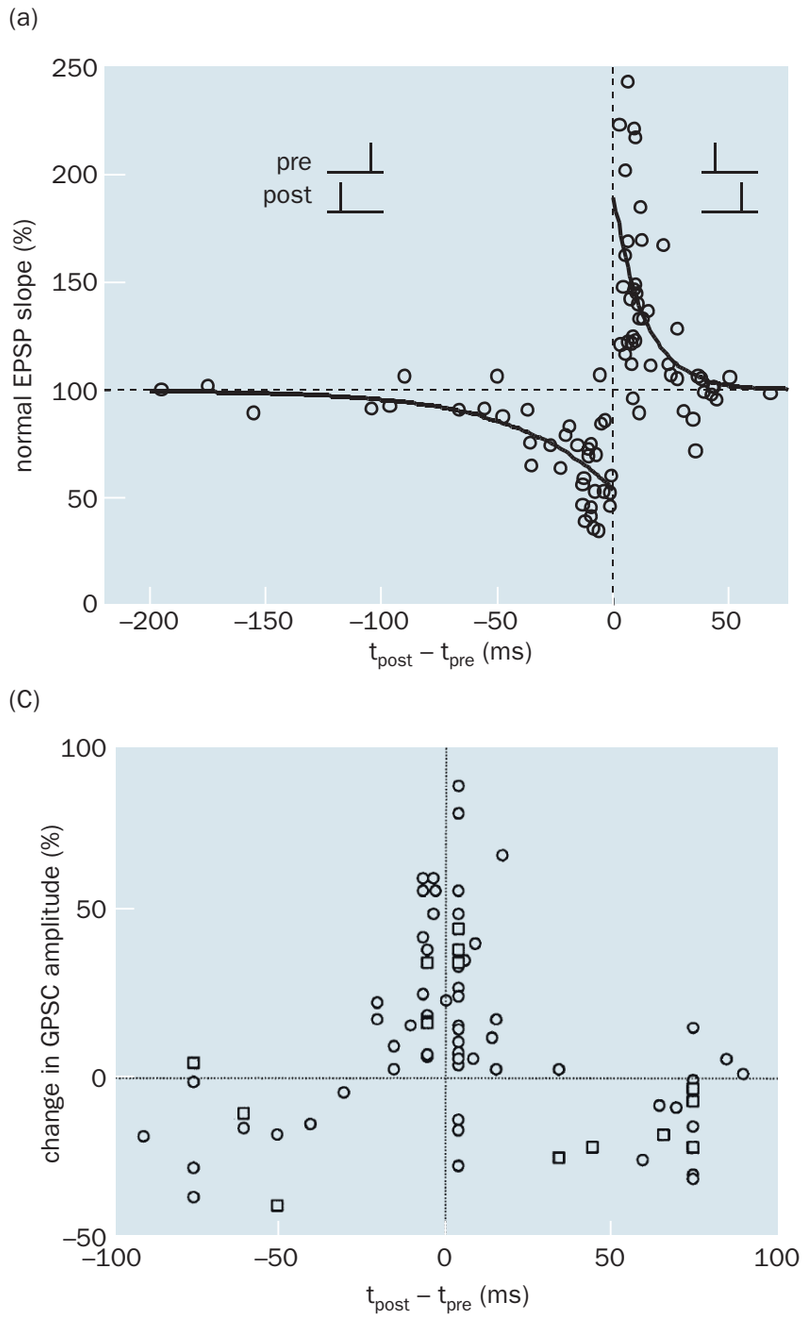


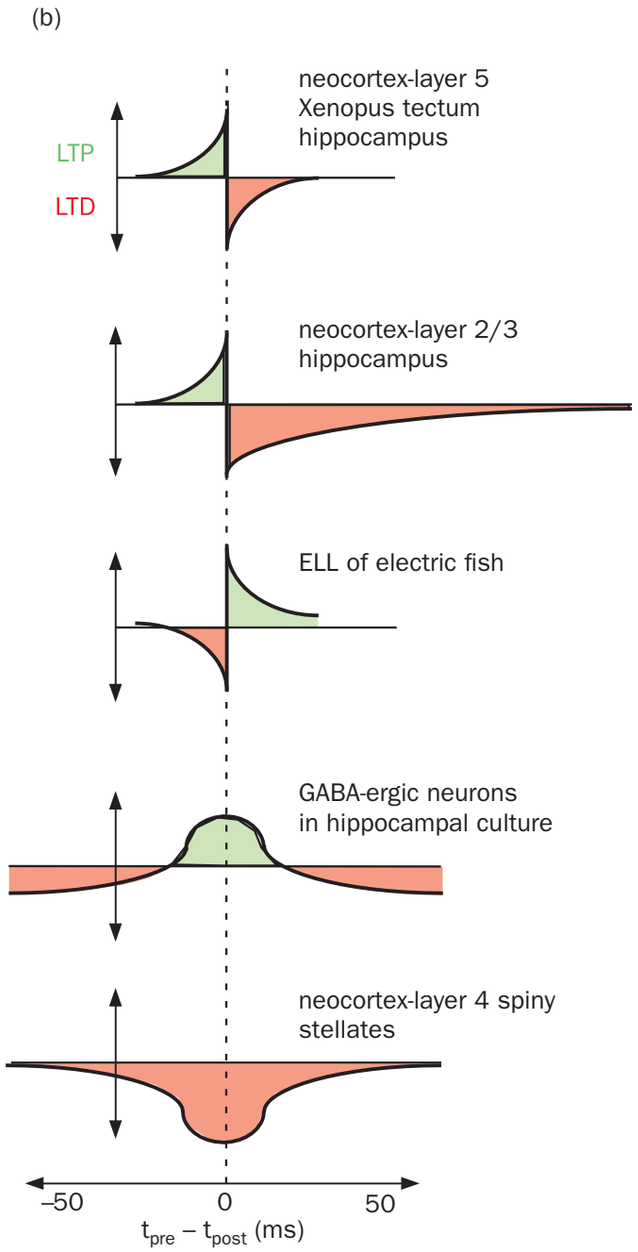




Markram et. al., 1997

Spike-timing-dependent plasticity

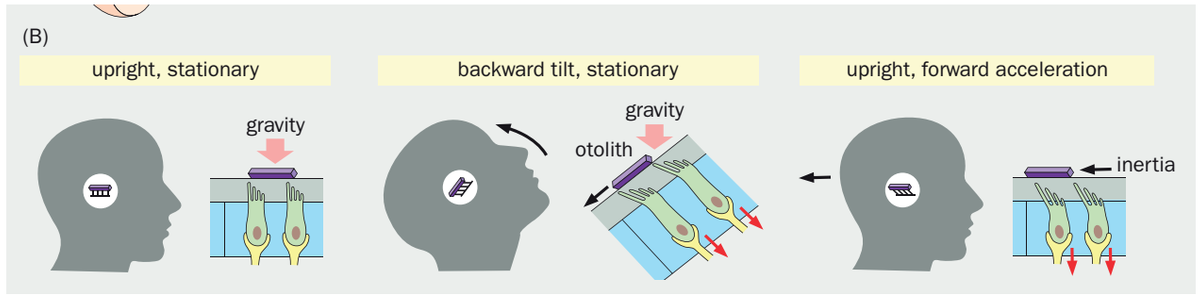




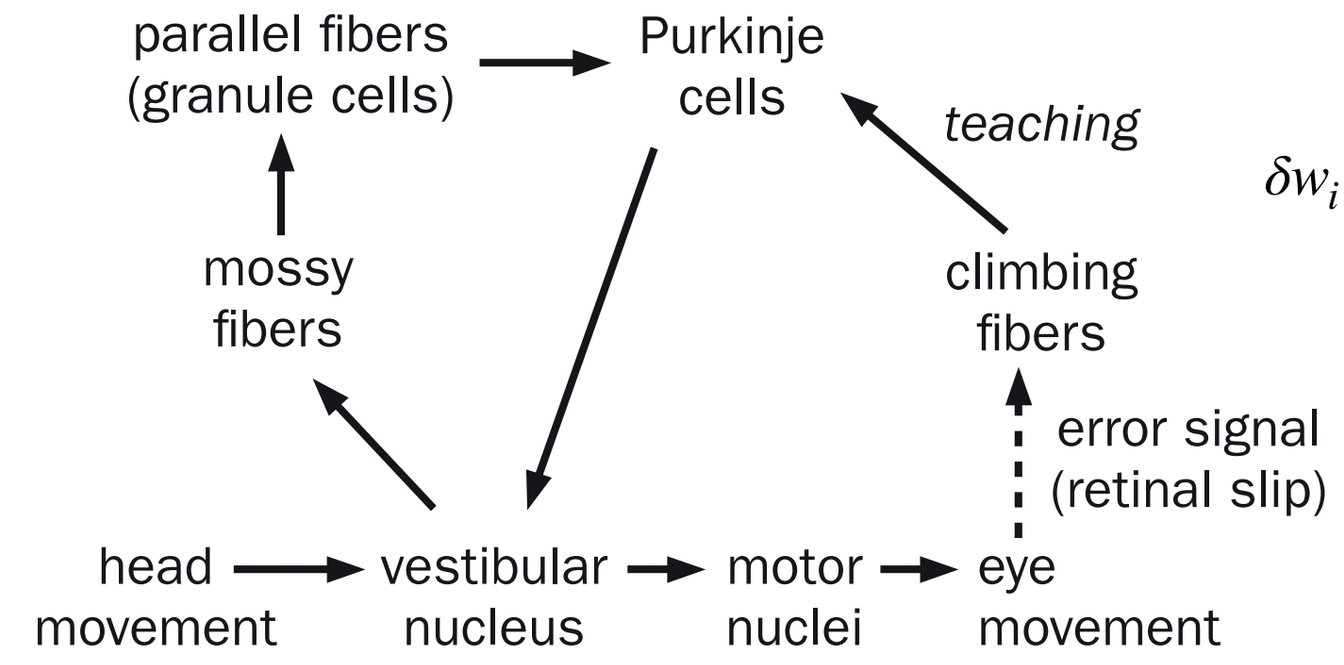
Bi & Poo, 1999, 2001, and others

Supervised learning in the cerebellum

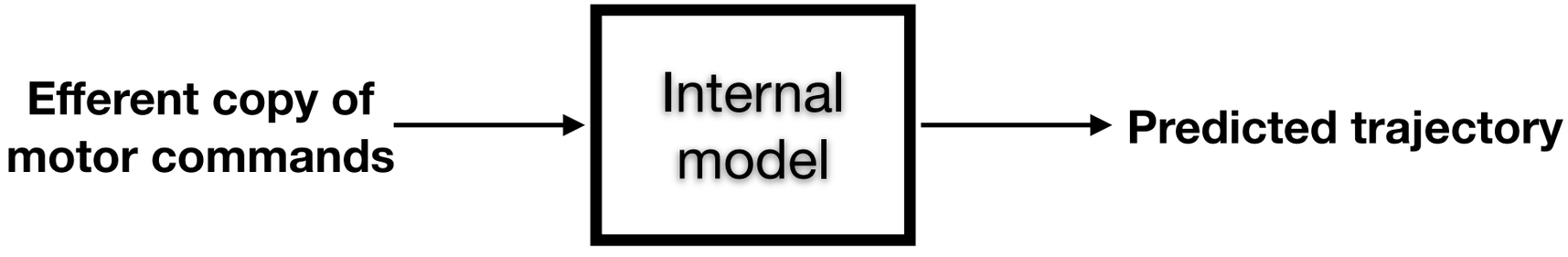
Cerebellar function in Vestibulo-ocular reflex



(A)

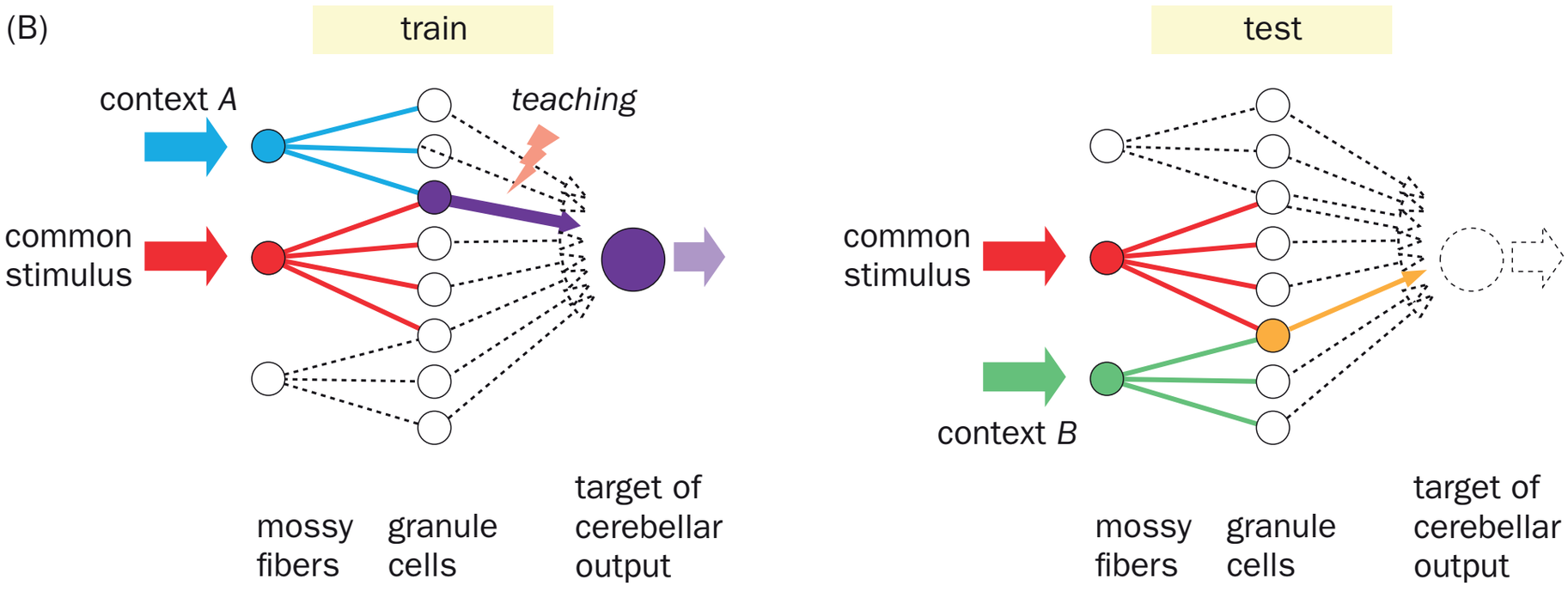


$$\delta w_i = -\epsilon(CF - \overline{CF})x_i$$

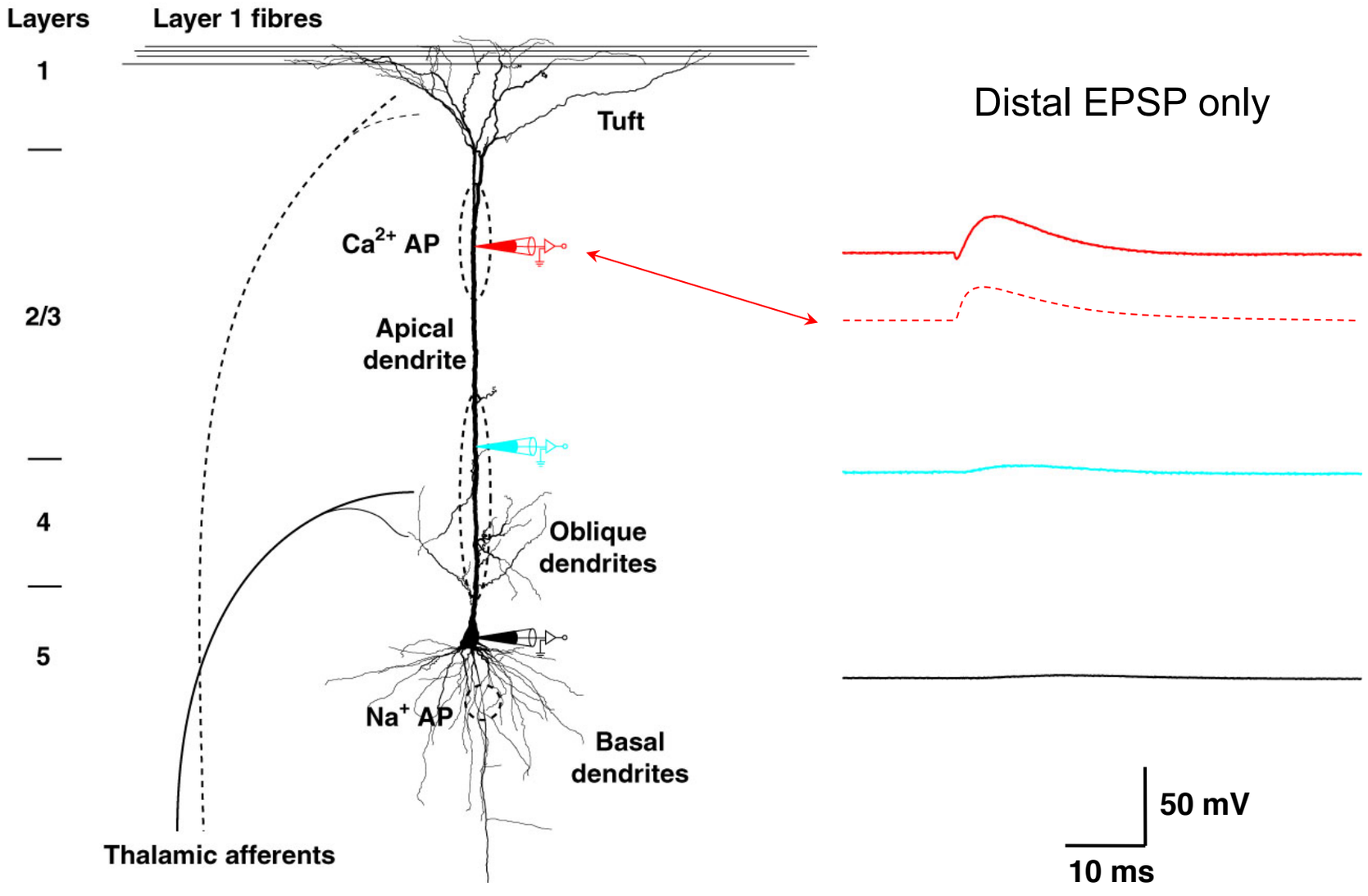


Masao Ito

Supervised learning in the cerebellum

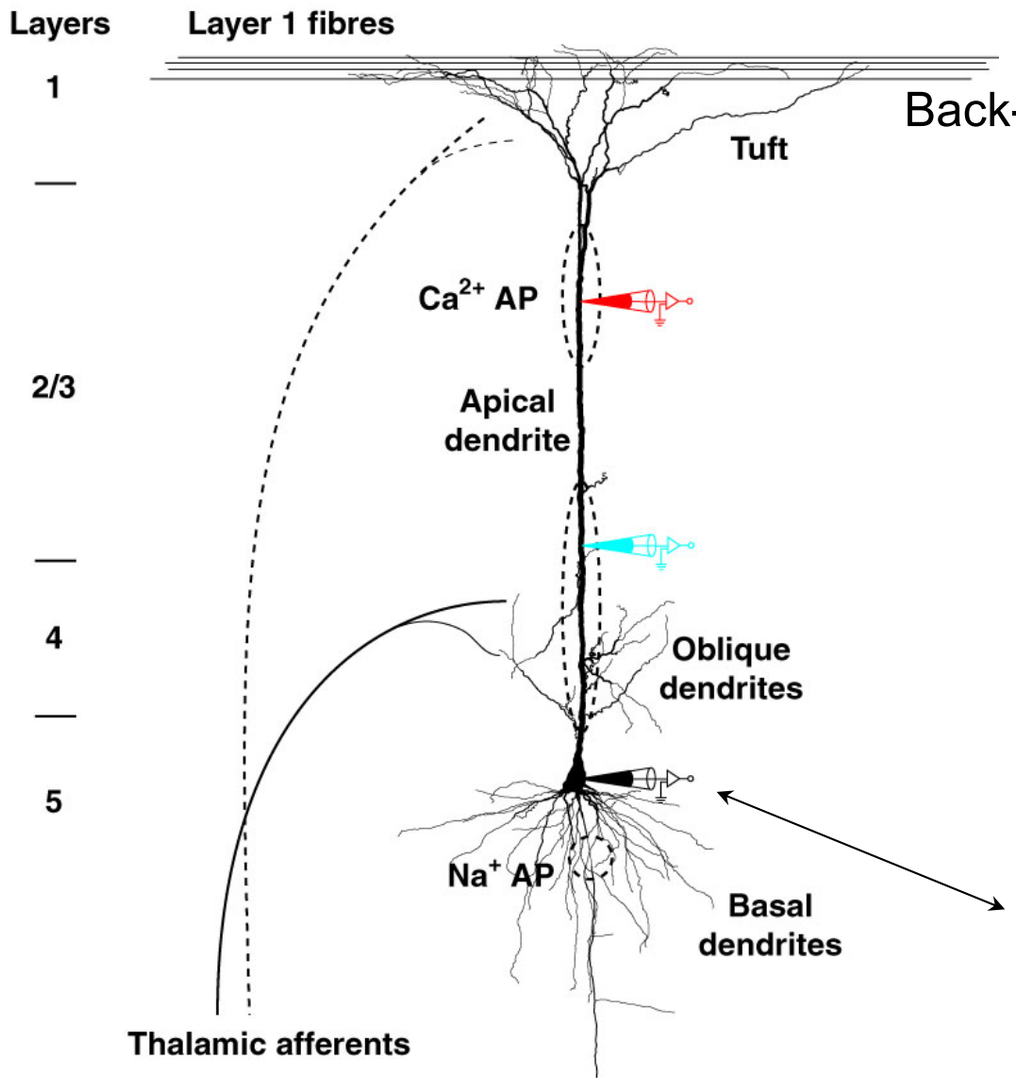


Dendritic Ca^{2+} plateau potential

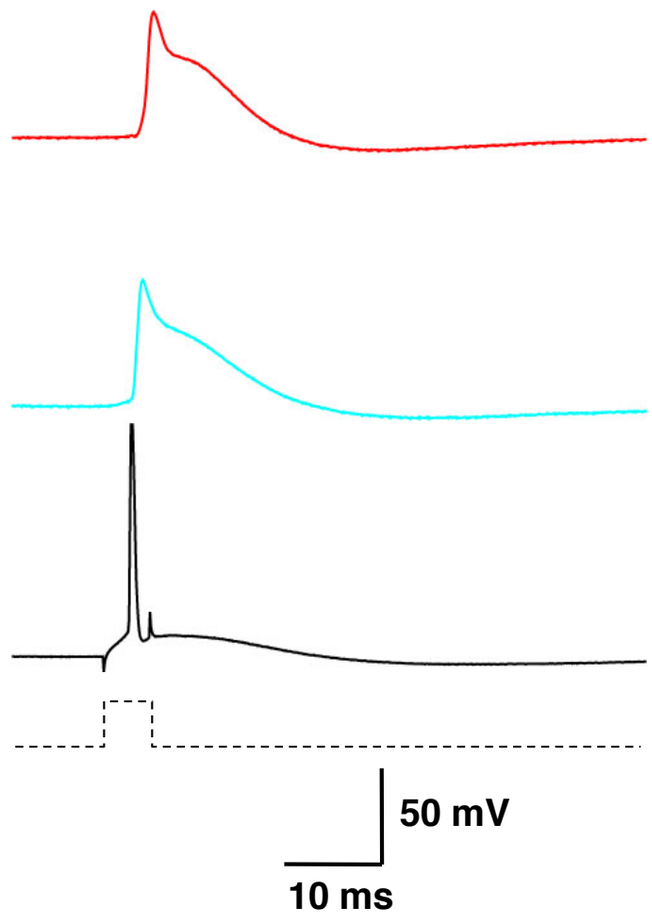


Larkum, Zhu & Sakmann - Nature, 1999

Dendritic Ca^{2+} plateau potential

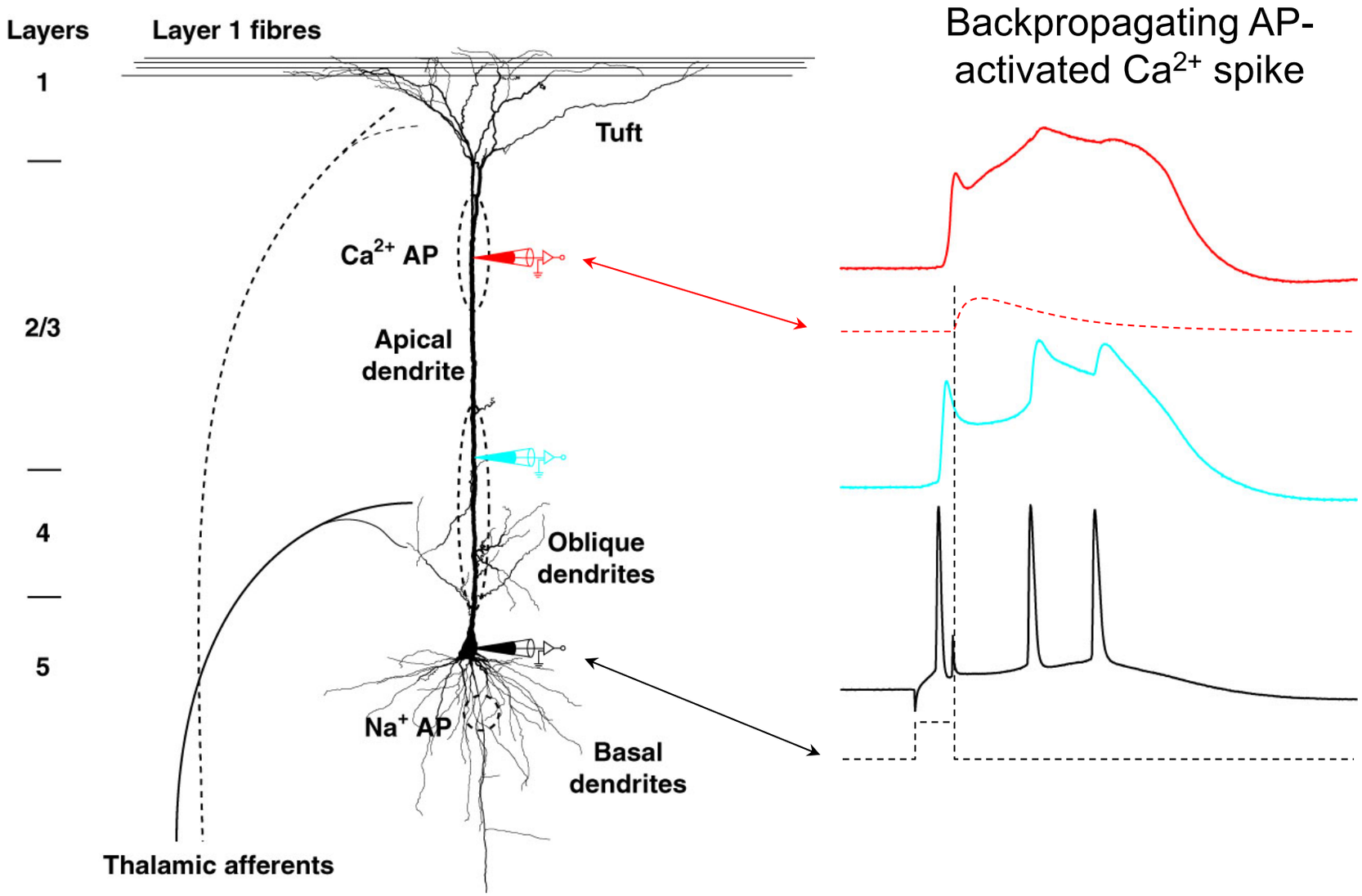


Back-propagating action potential only



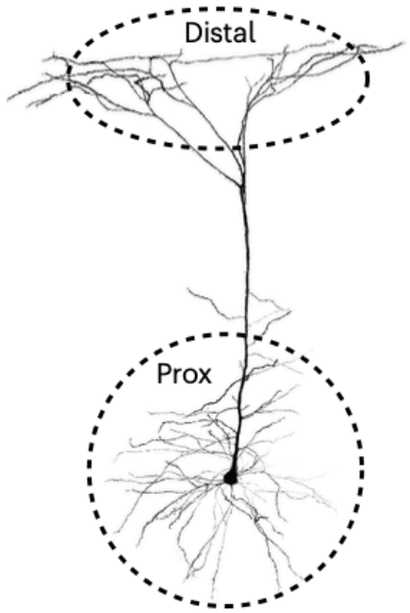
Larkum, Zhu & Sakmann - Nature, 1999

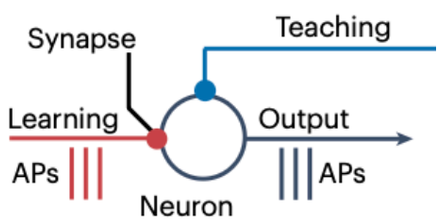
Dendritic Ca^{2+} plateau potential

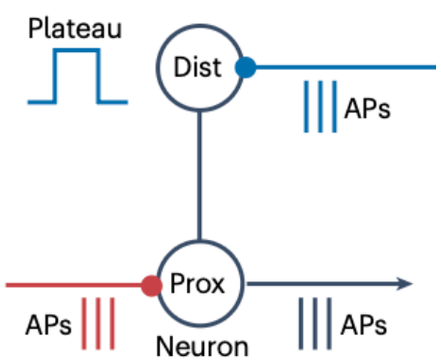


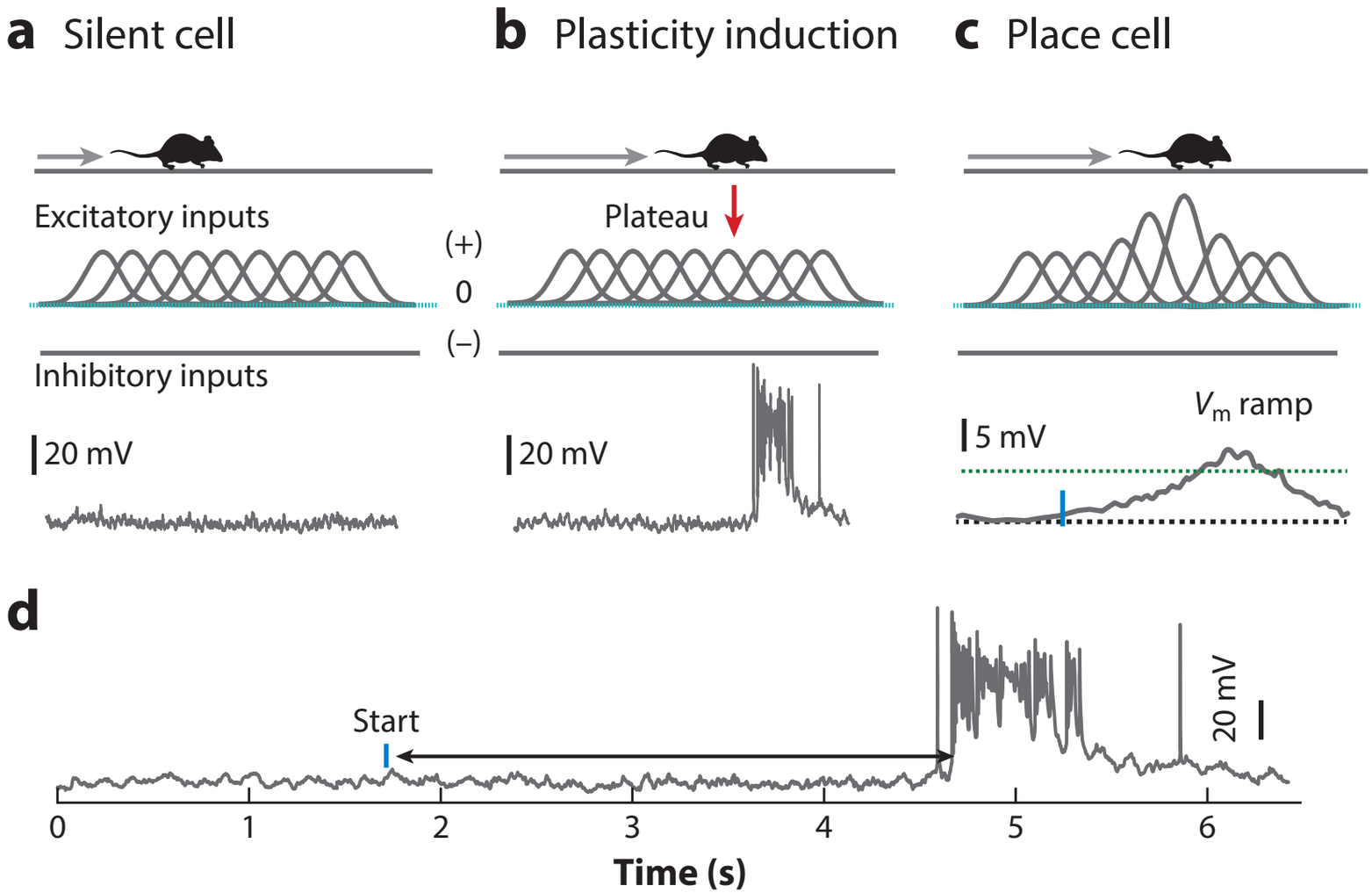
Larkum, Zhu & Sakmann - Nature, 1999

Behavior time-scale plasticity (BTSP) in Hippocampus

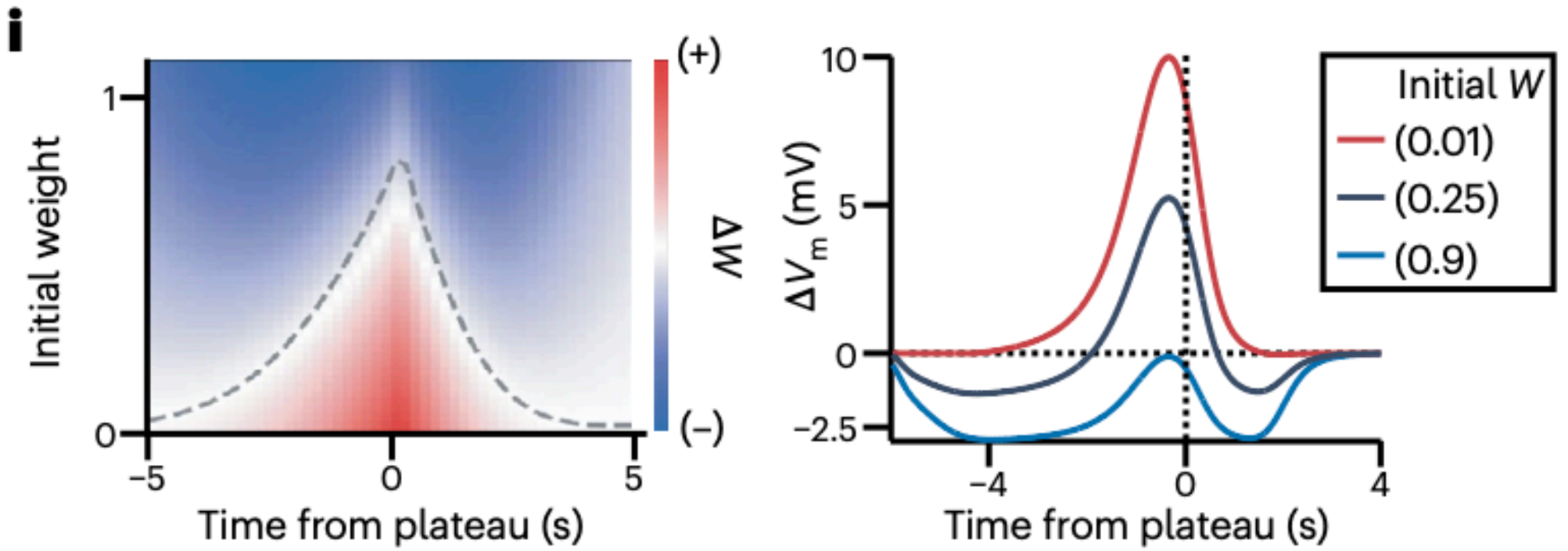




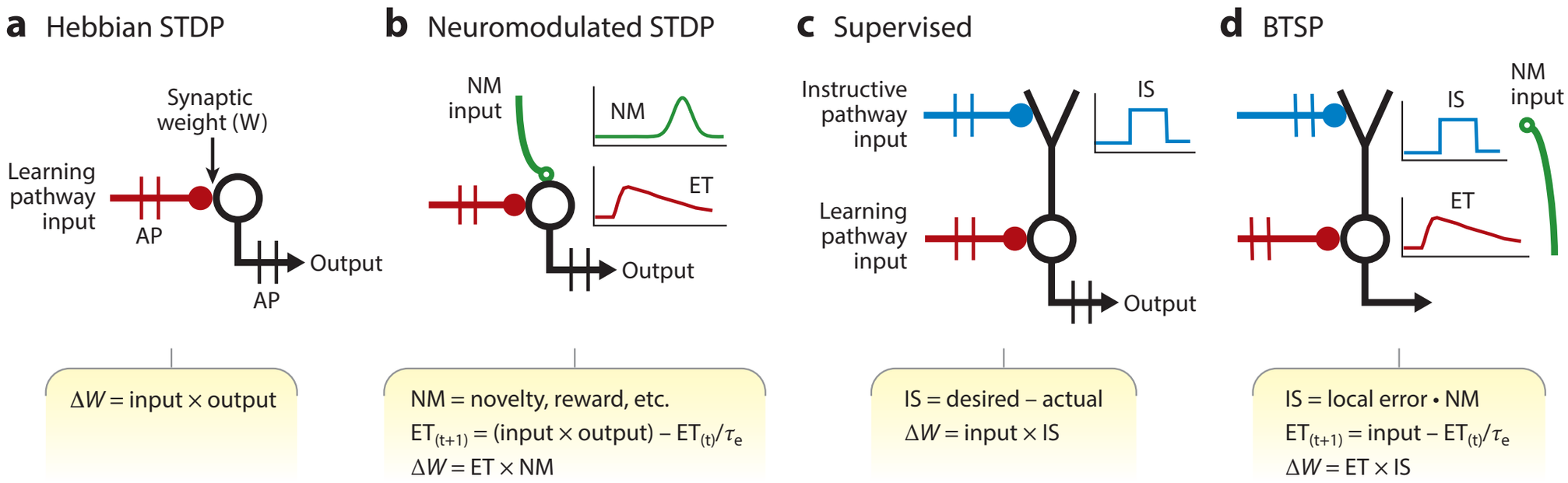




Behavior time-scale plasticity (BTSP) in Hippocampus



Synaptic learning rule (algorithms)



NM: neuromodulator

ET: eligibility trace (contingent vs non-contingent)

IS: instructive signal

Rich vs single learning rule

Table 1. The most commonly studied forms of plasticity all appear to be relevant to gradient estimation

Cell mechanisms	Relation to gradients
Hebbian plasticity	Gradient is zero for inactive cells
STDP	Gradient is zero if presynaptic spikes are too late
Probabilistic release	If release affects performance, gradient is non-zero
No action potential propagation into hyperpolarized segments	Gradient for synapses on a dendrite will be zero if a local dendrite prevents action potential propagation into its segments
CamKII	Recent concurrent pre- and postsynaptic activity indicates potential gradient signal
High-frequency bursts	Bursts can communicate strong relevance of a neuron to performance gradient
Circuit mechanisms	**Relation to gradients**
Dopamine	Gradients are large if we misestimate reward; dopamine can carry nature or error
Acetylcholine	Gradients should be zero if the situation is irrelevant
Intercellular messages (e.g. NO, RNA, peptides)	Neurons can communicate their contribution to gradients to other neurons using a variety of intercellular messenger systems
Climbing fibres in the cerebellum	Learning should be enabled by errors and can be used to estimate gradients
Attention	Only those parts of the brain that matter for the decision contribute to gradients

Richards & Kording 2023

$$w_{ji} \leftarrow w_{ji} - \eta \delta_j^l r_i^{l-1}$$

$$\delta_j^l = F'(I_j^l) \sum_k \delta_k^{l+1} w_{kj}$$

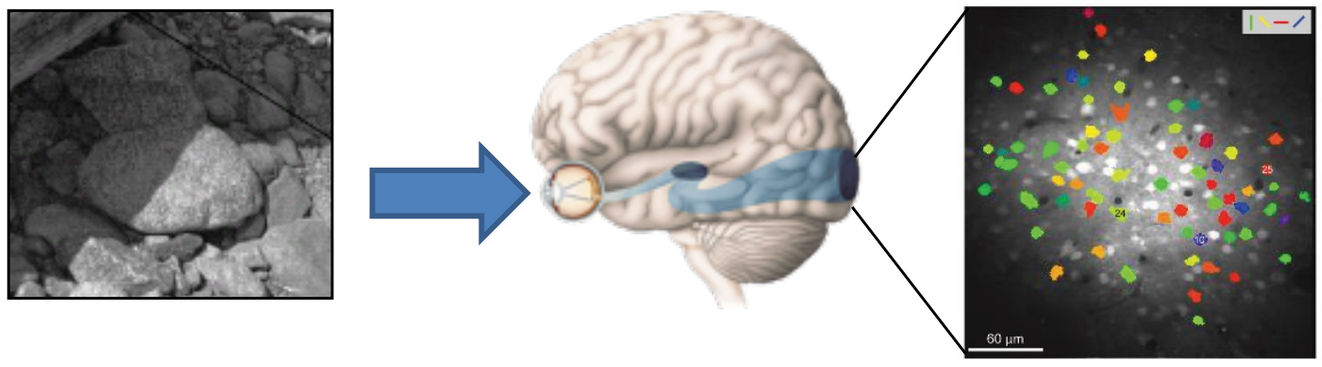
In different contexts, using different neurons to approximate gradients

Back-propagation

Hebbian Learning vs. Backpropagation

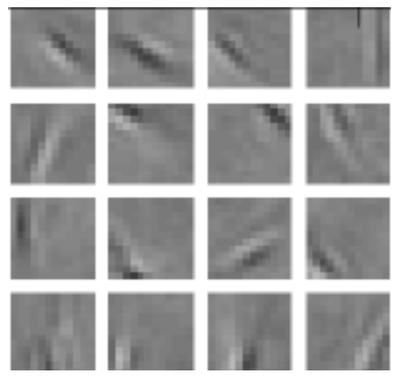
Feature	Hebbian / STDP	Backpropagation
Error Signal	?	Global loss gradient
Locality	Fully local	Non-local (weight transport)
Learning Type	?	Error-driven
Credit Assignment	?	Exact (chain rule)
Biological Plausibility	High	Low
Deep Network Training	Ineffective alone	Highly effective
Stability	Unstable (unbounded)	Stable (with regularization)

Reconstruction approach: linear decoding

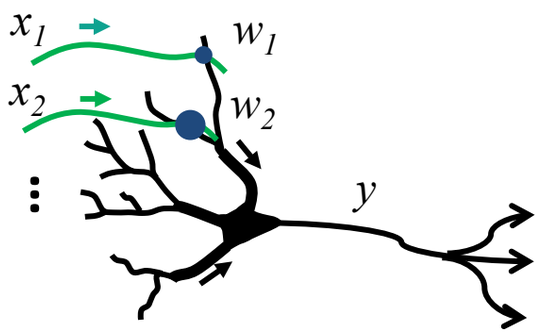


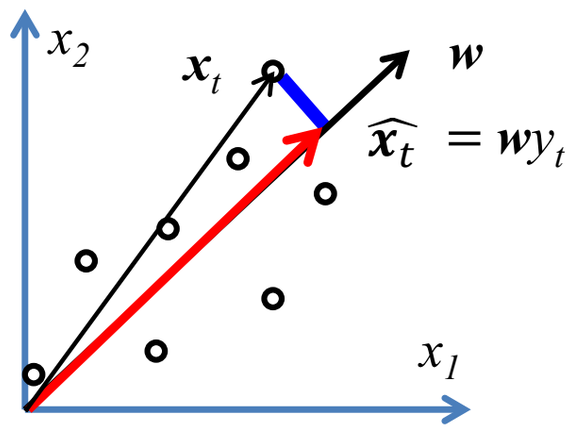
$$\mathbf{x}(t) \approx \mathbf{w_1} y_1 + \mathbf{w_2} y_2 + \ldots + \mathbf{w_k} y_k$$

$\mathbf{w}_{1:k}$



Reconstructing input signals from single neuron





Encoding:

$$y_t = \boldsymbol{w}^T \boldsymbol{x}_t \;, \|\boldsymbol{w}\|_2^2 = 1$$

Decoding:

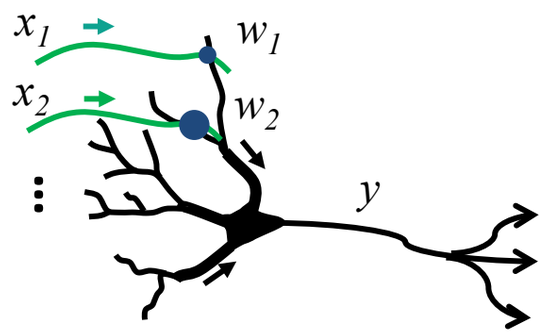
$$\widehat{\boldsymbol{x}_t} = \boldsymbol{w}y_t$$

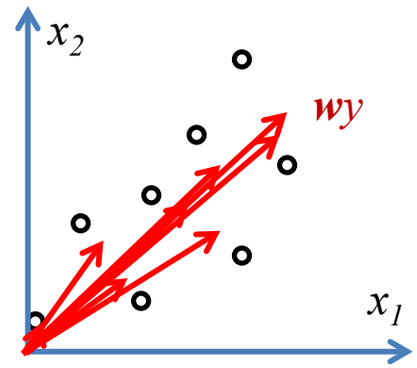
$$\min_{\widehat{\boldsymbol{x}_t}} \sum_{t=1}^{T} \|\boldsymbol{x}_t - \widehat{\boldsymbol{x}_t}\|_2^2 \qquad \min_{\boldsymbol{w}, y_t, \|\boldsymbol{w}\|=1} \sum_{t=1}^{T} \|\boldsymbol{x}_t - \boldsymbol{w}y_t\|_2^2$$

$$\operatorname*{argmin}_{\boldsymbol{w}, \|\boldsymbol{w}\|=1} \sum_{t=1}^{T} \|\boldsymbol{x}_t - \boldsymbol{w}\boldsymbol{w}^T\boldsymbol{x}_t\|_2^2 = \operatorname*{argmax}_{\boldsymbol{w}, \|\boldsymbol{w}\|=1} \sum_{t=1}^{T} \boldsymbol{w}^T\boldsymbol{x}_t\boldsymbol{x}_t^T\boldsymbol{w}$$

$$= \operatorname*{argmax}_{\boldsymbol{w}, \|\boldsymbol{w}\|=1} \boldsymbol{w}^T \sum_{t=1}^{T} \boldsymbol{x}_t\boldsymbol{x}_t^T\boldsymbol{w} = \mathrm{eig}_1\left(\sum_{t}^{T} \boldsymbol{x}_t\boldsymbol{x}_t^T\right)$$

Online dimensionality reduction
top principal component



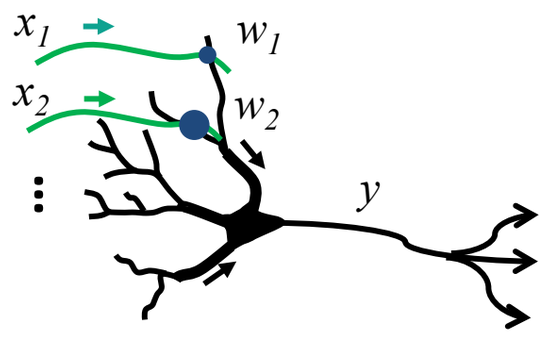


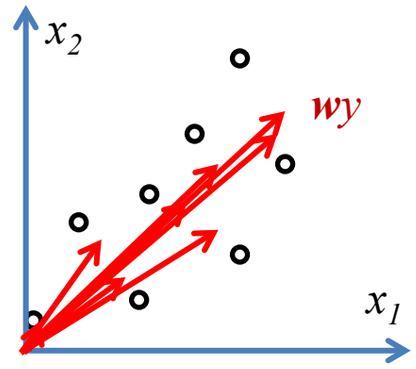
$$\min_{\boldsymbol{w},\|\boldsymbol{w}\|=1} \sum_{t=1}^{T} \min_{y_t} \|\boldsymbol{x}_t - \boldsymbol{w} y_t\|_2^2$$

Phase 1: minimize neural activity

$$y_t = \boldsymbol{w}_{t-1}^{\mathrm{T}} \boldsymbol{x}_t$$

Online dimensionality reduction
top principal component





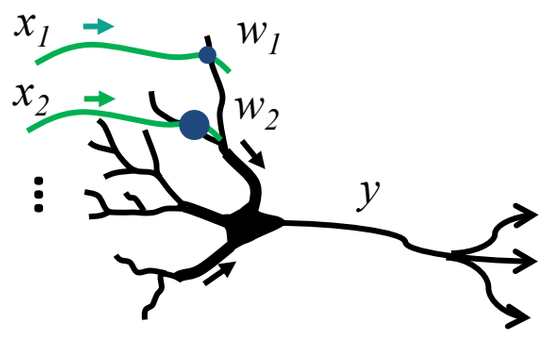
$$\min_{\boldsymbol{w},\|\boldsymbol{w}\|=1} \sum_{t=1}^{T} \min_{y_t} \|\boldsymbol{x}_t - \boldsymbol{w}y_t\|_2^2$$

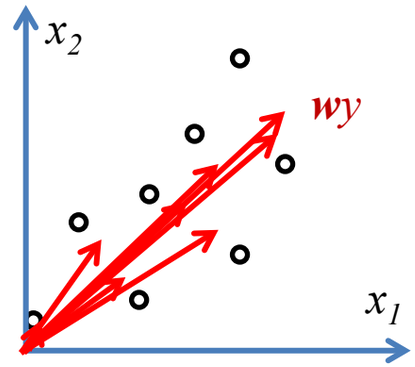
Phase 2: minimize synaptic weight

$$\mathbf{w}_T = \arg\min_{\mathbf{w}} \sum_{t=1}^{T} \left\|\mathbf{x}_t - \mathbf{w}y_t\right\|_2^2$$

$$= \arg\min_{\mathbf{w}} \sum_{t=1}^{T} \left[-\mathbf{w}^\top \mathbf{x}_t y_t + \left\|\mathbf{w}\right\|_2^2 y_t^2 \right] \quad = \frac{\sum_{t=1}^{T} \mathbf{x}_t y_t}{\sum_{t=1}^{T} y_t^2}$$

Online dimensionality reduction
top principal component





$$\min_{\boldsymbol{w},\|\boldsymbol{w}\|=1} \sum_{t=1}^{T} \min_{y_t} \|\boldsymbol{x}_t - \boldsymbol{w}y_t\|_2^2$$

Phase 2: minimize synaptic weight

$$\mathbf{w}_T = \mathbf{w}_{T-1} + y_T \left(\mathbf{x}_T - \mathbf{w}_{T-1} y_T \right) \Big/ \sum_{t=1}^{T} y_t^2$$

Hebbian plasticity and Oja rule

neural activity: $y_t = \boldsymbol{w}_{t-1}^{\mathrm{T}} \boldsymbol{x}_t$

synaptic weight: $\mathbf{w}_T = \mathbf{w}_{T-1} + y_T \left(\mathbf{x}_T - \mathbf{w}_{T-1} y_T \right) \Big/ \sum_{t=1}^{T} y_t^2$

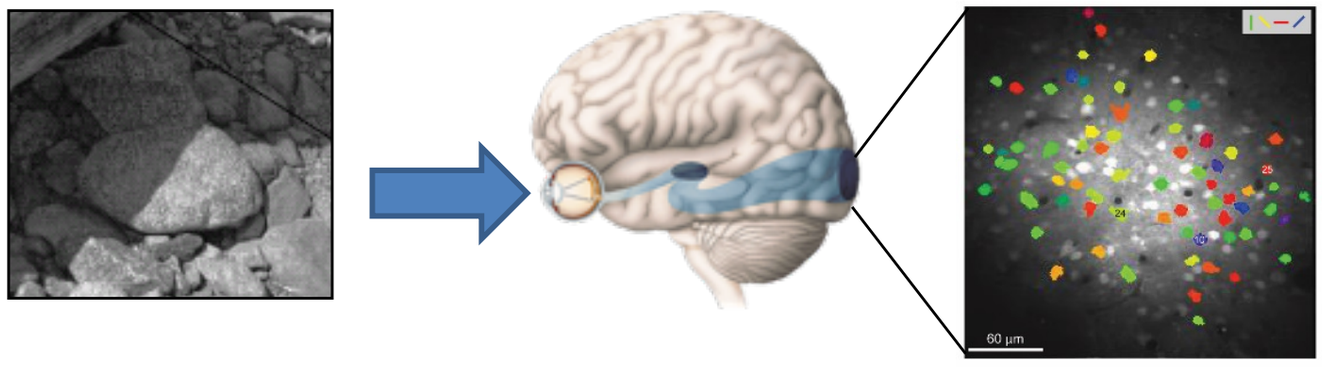
$$\tau_w \frac{d\mathbf{w}}{dt} = y\mathbf{x} - y^2\mathbf{w},$$

$$\tau_w \frac{d\|\mathbf{w}\|^2}{dt} = 2\tau_w \mathbf{w} \cdot \frac{d\mathbf{w}}{dt} = 2y^2(1 - \|\mathbf{w}\|^2)$$

synaptic weight is bounded!

$\mathbf{x} \in \mathbb{R}^n$ $\mathbf{y} \in \mathbb{R}^k$

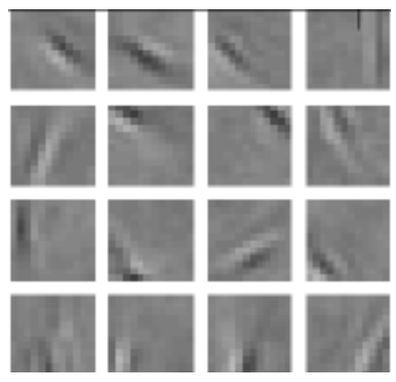
Natural Scenes Neural Activity



$$\mathbf{x}(t) \approx \mathbf{w_1} y_1(t) + \mathbf{w_2} y_2(t) + \ldots + \mathbf{w_k} y_k(t)$$

$$\mathbf{x}(t) \approx \mathbf{W}\mathbf{y}(t)$$

$\mathbf{w}_{1:k}$

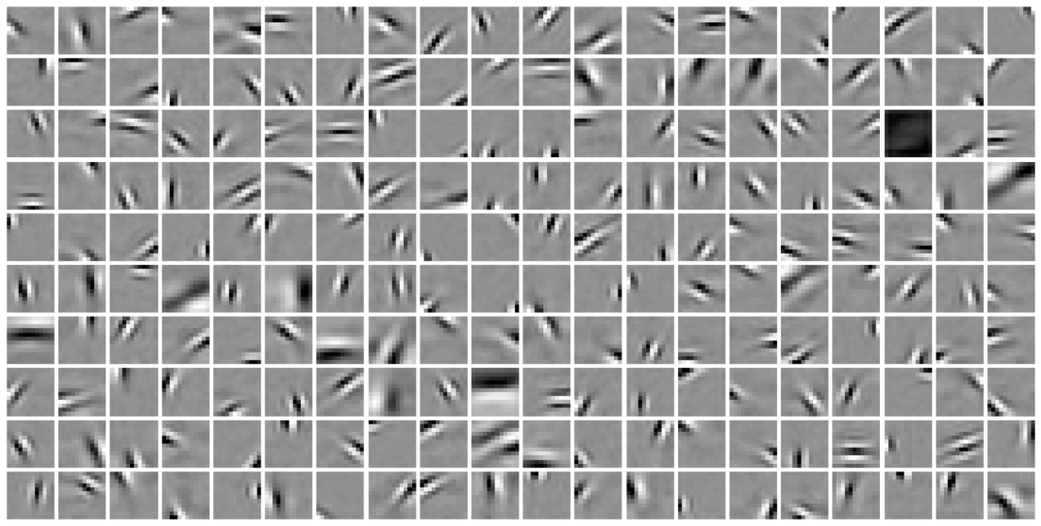


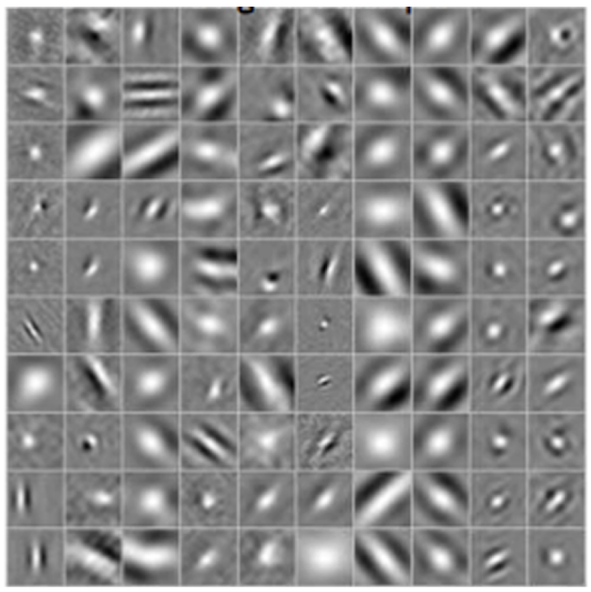
Efficient reconstruction theory: $\min_{\mathbf{W}} \sum_t \min_{\mathbf{y_t}} \|\mathbf{x_t} - \mathbf{W}\mathbf{y_t}\|_2^2 + \text{constraints}$

Sparse coding

Olshausen & Field (1996): Sparse, overcomplete (k > n) representations

$$\min_{\mathbf{W}} \sum_{t} \min_{\mathbf{y_t}} \|\mathbf{x_t} - \mathbf{W}\mathbf{y_t}\|_2^2 + \lambda \sum_{t} \|\mathbf{y_t}\|_1$$

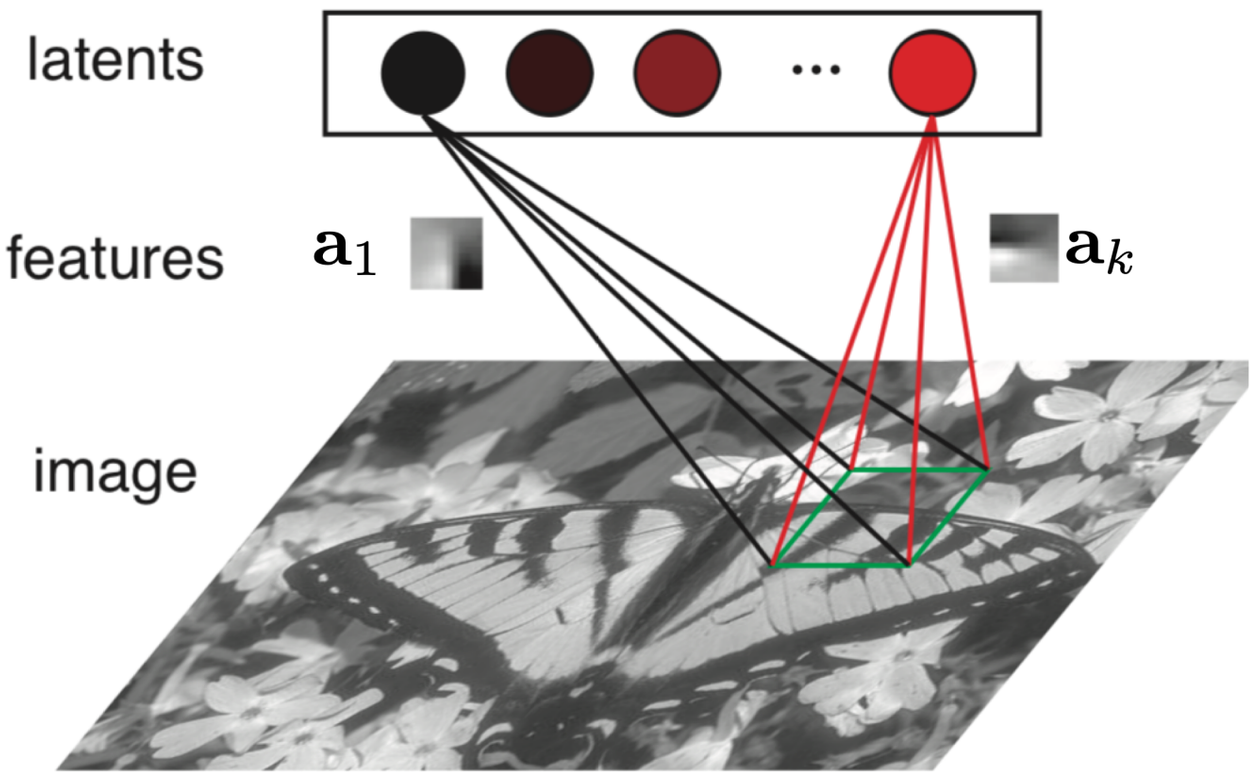




Data from Ringach lab

Probabilistic interpretation of Sparse Coding

Sparse coding is based on a “generative model”. Task is to recover “latent” generating factors.



$$\mathbf{y}_t \in \mathbb{R}^k$$

Observed

$$\mathbf{x}_t = \mathbf{W}\mathbf{y}_t + \eta$$

$$\mathbf{x}_t \in \mathbb{R}^n$$

How do we infer the latent factors from the observed variables? For n < k, this is a terribly ill-defined problem. Need to assume something! Priors!

Figure modified from Barello, Charles, Pillow 2018

$$\mathbf{x}_t = \mathbf{W}\mathbf{y}_t + \eta \qquad \mathbf{x}_t \in \mathbb{R}^n, \quad \mathbf{y}_t \in \mathbb{R}^k \qquad k > n$$

Prior: $p(\mathbf{y}_t) \propto e^{-\lambda|\mathbf{y}_t|_1}$ Conditional: $p(\mathbf{x}_t \,|\, \mathbf{y}_t;\ \mathbf{W}) \propto e^{-\frac{|\mathbf{x} - \mathbf{W}\mathbf{y}_t|^2}{2\sigma_\eta^2}}$

Model likelihood: $p(\mathbf{x}_t; \mathbf{W}) = \int d\mathbf{y}_t p(\mathbf{x}_t \,|\, \mathbf{y}_t; \mathbf{W})\ p(\mathbf{y}_t)$

$$\begin{aligned}
\max_{\mathbf{W}} \log p(\mathbf{x}_t; \mathbf{W}) &= \max_{\mathbf{W}} \log \int d\mathbf{y}_t p(\mathbf{x}_t \,|\, \mathbf{y}_t; \mathbf{W}) p(\mathbf{y}_t) \\
&\approx \max_{\mathbf{W}} \log \max_{\mathbf{y}_t} p(\mathbf{x}_t \,|\, \mathbf{y}_t; \mathbf{W}) p(\mathbf{y}_t) \\
&= \max_{\mathbf{W}} \max_{\mathbf{y}_t} \log p(\mathbf{x}_t \,|\, \mathbf{y}_t; \mathbf{W}) p(\mathbf{y}_t) \\
&= \max_{\mathbf{W}} \max_{\mathbf{y}_t} -\frac{|\mathbf{x}_t - \mathbf{W}\mathbf{y}_t|^2}{2\sigma_\eta^2} - \lambda |\mathbf{y}_t|_1 + \ldots
\end{aligned}$$

$$\mathbf{x}_t = \mathbf{W}\mathbf{y}_t + \eta \qquad\qquad \mathbf{x}_t \in \mathbb{R}^n, \quad \mathbf{y}_t \in \mathbb{R}^k$$

$$\max_{\mathbf{W}} \max_{\mathbf{y}_t} - \frac{|\mathbf{x}_t - \mathbf{W}\mathbf{y}_t|^2}{2\sigma_\eta^2} - \lambda |\mathbf{y}_t|_1 + \ldots$$

Update
Generative
Model
(Plasticity)

Infer latent
Variables
(Neural activity)

The cost function

$$E = \|\mathbf{x} - \mathbf{W}\mathbf{y}\|_2^2 + \lambda_2 g(\mathbf{y}) + \lambda \sum_{i,j} W_{ij}^2$$

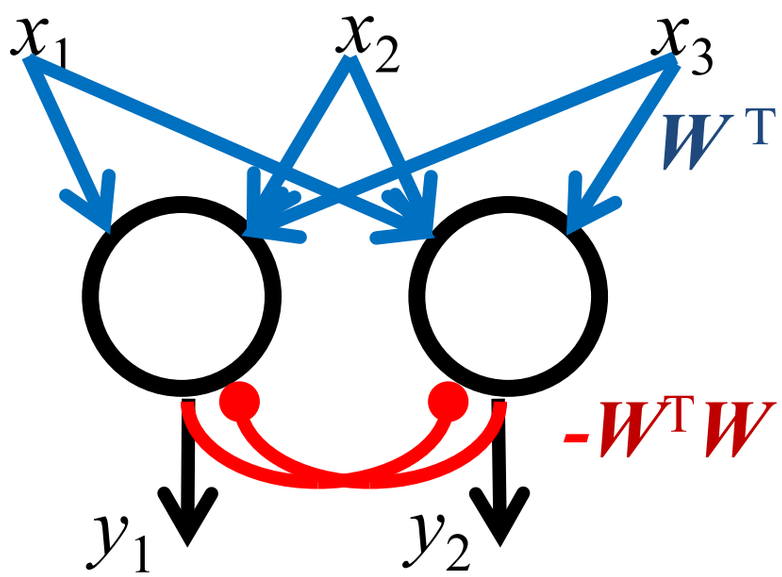
$$g(\mathbf{y}) = \|\mathbf{y}\|_2^2 \text{ gaussian prior}$$

$$= \sum_i \log(1 + y_i^2) \text{ sparse prior}$$

neural activity: $$\frac{d\mathbf{y}}{dt} = -\frac{\partial E}{\partial \mathbf{y}}$$

synaptic weight: $$\frac{d\mathbf{W}}{dt} = -\frac{\partial E}{\partial \mathbf{W}}$$

Neural circuit implementation (single layer)



$$\frac{d\mathbf{y}}{dt} = -\lambda_2\mathbf{y} + W^T\mathbf{x} - W^TW\mathbf{y}$$

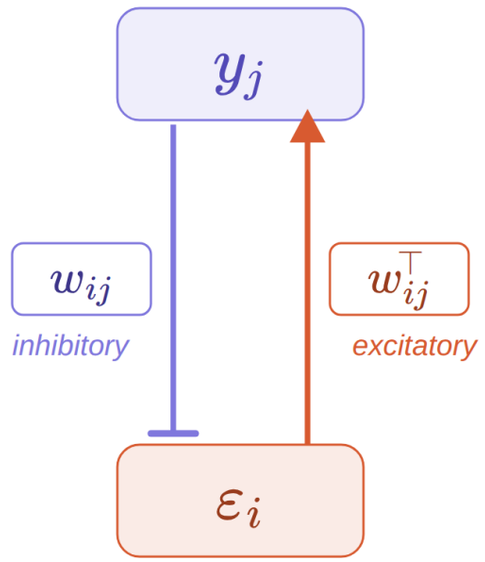
HW: Derive this!

$$\frac{dW_{ij}}{dt} = -\lambda W_{ij} + x_i y_j - y_j \sum_k W_{ik} y_k$$

Non-local learning rule!

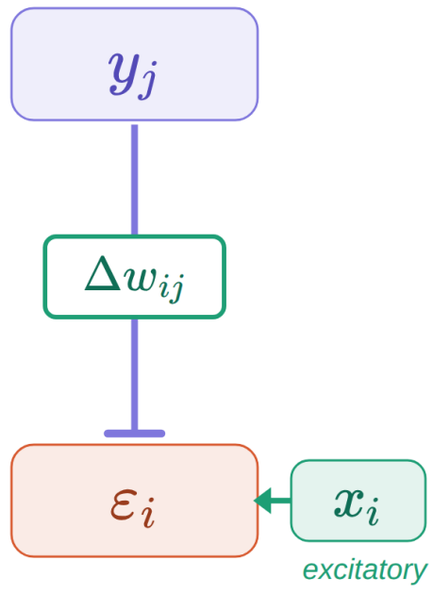
Predictive coding (two layers)

Phase 1 — Inference



Same weight used in both directions — ⚠ weight transport problem

Phase 2 — Weight update



Local update: postsynaptic ε_i × presynaptic y_j

▲ Excitatory — Inhibitory

Inference dynamics

$$\varepsilon_i = x_i - \sum_k w_{ik}\, y_k$$

$$\dot{y}_j = \sum_i w_{ij}\, \varepsilon_i - y_j$$

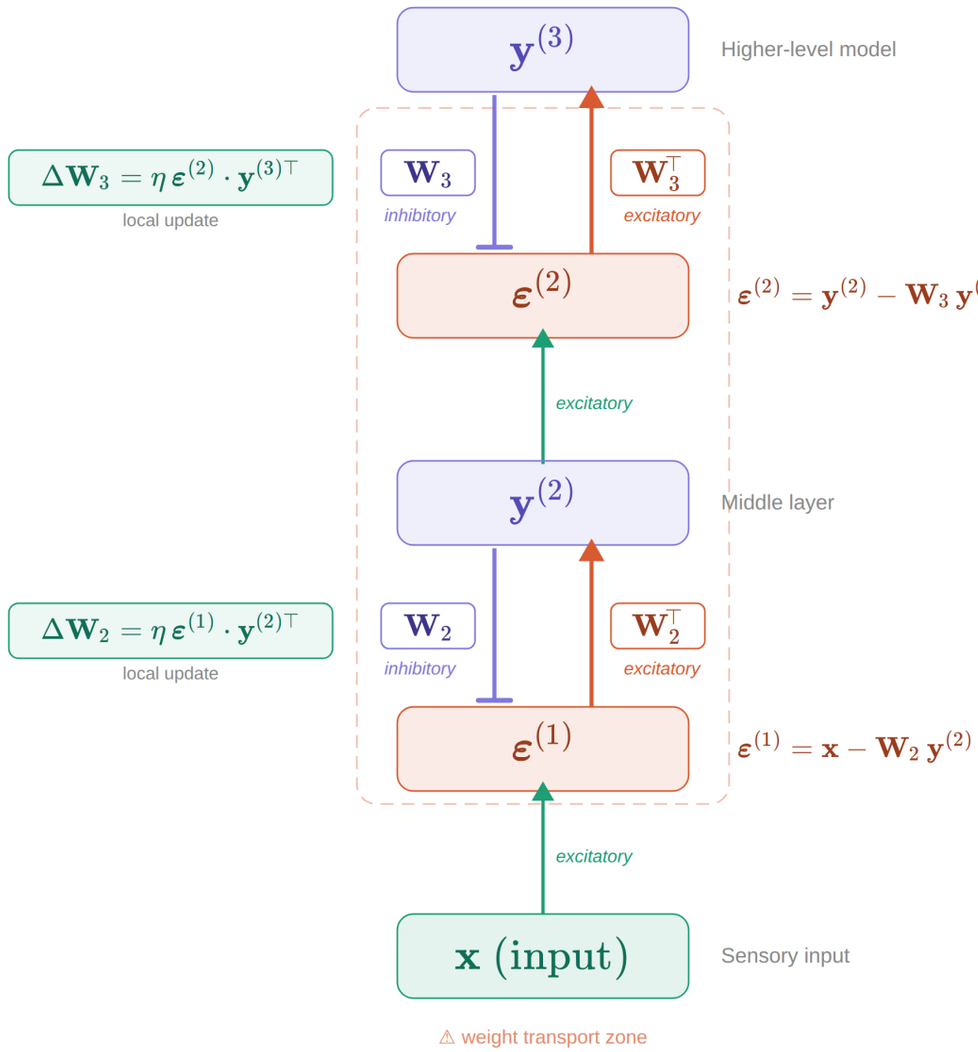
Error propagates back up via $\mathbf{W}^\top$ — requires weight transport

Synaptic plasticity

$$\Delta w_{ij} = \eta \cdot \varepsilon_i \cdot y_j$$

Purely local: postsynaptic ε_i × presynaptic y_j

Hierarchical predictive coding



Excitatory — Inhibitory — Weight update (local) — Weight transport zone

Inference (Phase 1)

$$\boldsymbol{\varepsilon}^{(\ell)} = \mathbf{y}^{(\ell)} - \mathbf{W}_{\ell+1}\, \mathbf{y}^{(\ell+1)}$$

$$\dot{\mathbf{y}}^{(\ell)} = -\boldsymbol{\varepsilon}^{(\ell)} + \mathbf{W}_\ell^\top\, \boldsymbol{\varepsilon}^{(\ell-1)} - \mathbf{y}^{(\ell)}$$

Each layer settles by integrating top-down errors via $\mathbf{W}_\ell^\top$

Plasticity (Phase 2)

$$\Delta \mathbf{W}_\ell = \eta \cdot \boldsymbol{\varepsilon}^{(\ell-1)} \cdot \mathbf{y}^{(\ell)\top}$$

Each synapse updates from local error × local activity — no transpose needed

⚠ Weight transport

The backward error pathways $\mathbf{W}_\ell^\top$ must mirror the forward prediction pathways $\mathbf{W}_\ell$ — two separate physical synapses that must remain identical

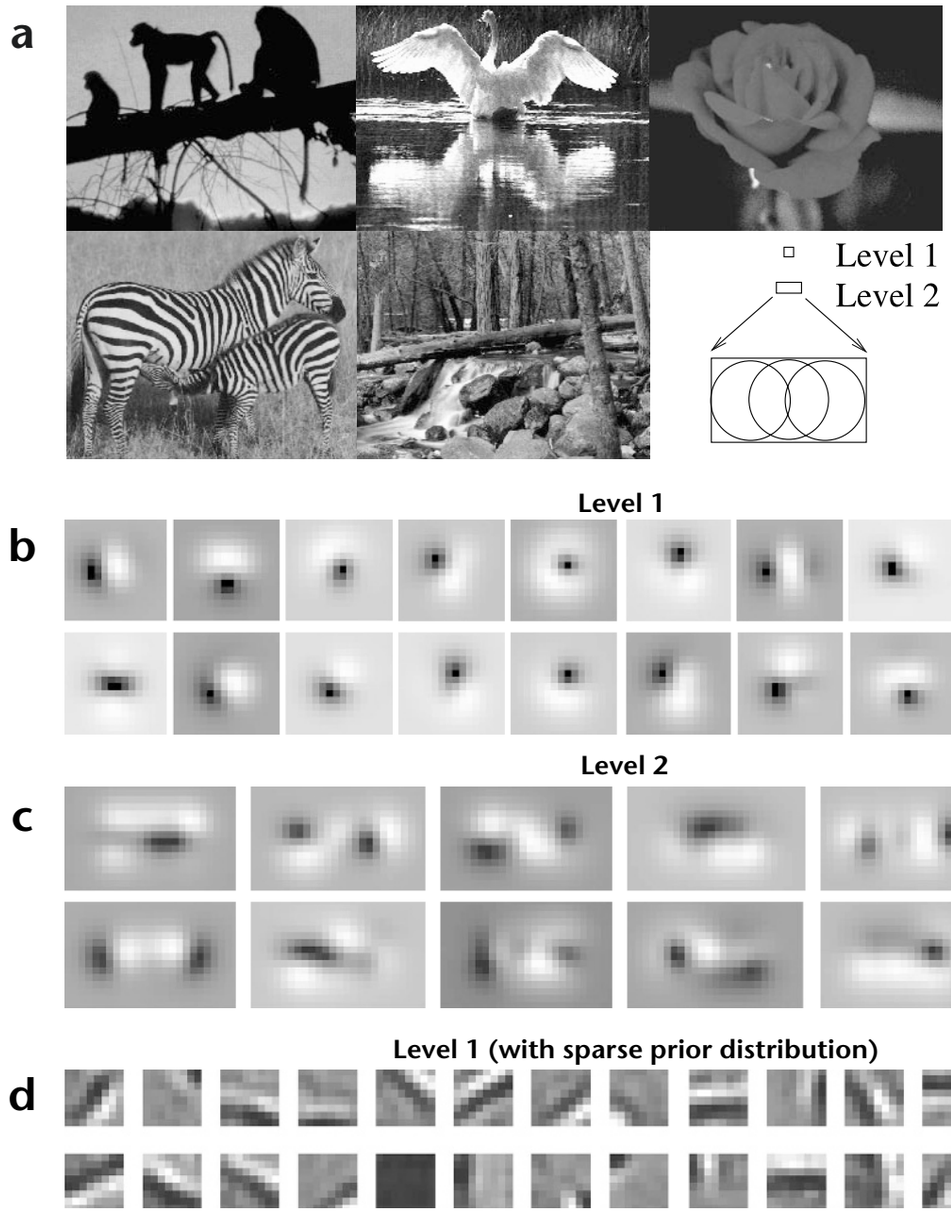
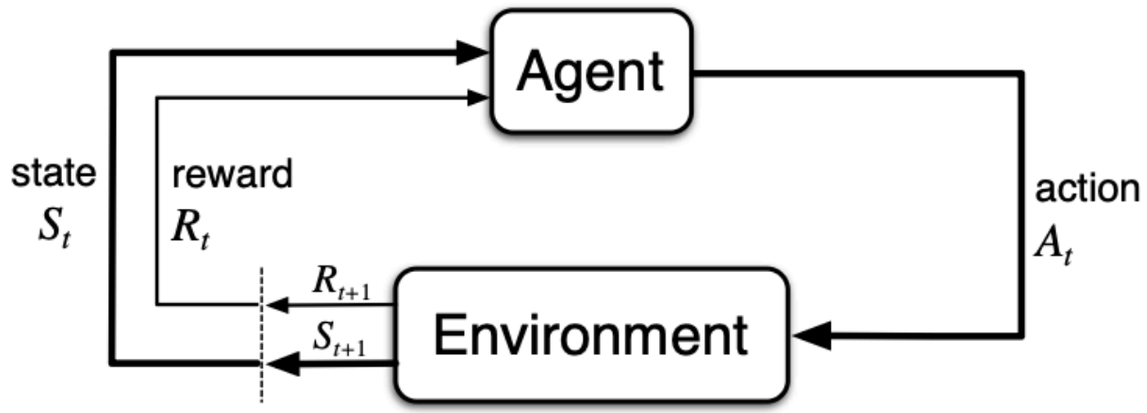


Fig. 2. Receptive fields of feedforward model neurons after training on natural images. **(a)** Five natural images used for training the three-level hierarchical network of Fig. 1c (Methods). The two upper boxes in the bottom right corner show relative sizes (16 x 16 and 16 x 26 pixels) of level-1 and level-2 receptive fields respectively. **(b)** Learned synaptic weights (RF weighting profiles) of 20 of the 32 feedforward model neurons in the level-1 module analyzing the central image region. Flanking image regions were analyzed by two other level-1 modules (**Fig. 1c**), each with 32 feedforward model neurons (Methods). Values for these synapses, which form rows of the matrix U^T, can be positive (excitatory, bright regions) or negative (inhibitory, dark regions). These RF profiles resemble classical oriented-edge/bar detectors characteristic of simple cells[2]. **(c)** RF profiles of 12 of the 128 level-2 feedforward model neurons. **(d)** Localized RF profiles resembling Gabor wavelets obtained by using a sigmoidal nonlinearity in the generative model, along with a sparse kurtotic prior distribution for the network activities (Methods). All 32 level-1 feedforward model neurons are shown; Gaussian windowing of inputs (as in b) was not necessary in this case.

Learning Rules II: Reward and Reinforcement learning

The reinforcement learning setting

Trial and error learning in sequential tasks, where choices lead to more choices



- An agent interacts with an environment over time
- At each time step t: observe state S_t, take action A_t, receive reward R_t
- Goal: maximize expected cumulative future reward
- This expected sum is called the "value function"

The value function

How much total future reward do I expect from state s?

$$V(s) = \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \cdots \,\middle|\, s_t = s\right]$$

$V(s)$	value of state s — expected cumulative discounted reward
r_t	reward received at time t
$\gamma \in [0,1)$	discount factor — how much future rewards are worth relative to now
$\mathbb{E}[\ldots]$	expectation over future states and actions

The Bellman equation

The value function satisfies a recursive relationship:

$$V(s) = \mathbb{E}\left[r + \gamma V(s')\right]$$

The value of being in state s equals the immediate reward plus the discounted value of the next state

Key insight: value is defined in terms of itself

→ This recursion is the foundation of all RL algorithms

The Q function (action-value)

To choose actions, we need to know the value of each action in each state:

$$Q(s,a) = \mathbb{E}\left[r + \gamma \max_{a'} Q(s',a') \middle| s_t = s, a_t = a \right]$$

Q(s, a) = expected cumulative reward from taking action a in state s, then acting optimally afterwards

Optimal policy: $\pi^*(s) = \arg\max_a Q(s,a)$

→ If we know Q, decision-making reduces to a simple comparison

Why is computing value hard?

$$Q(s_t, a_t) = r(s_t) + \sum_{s_{t+1}} P(s_{t+1} \mid s_t, a_t) \left[r(s_{t+1}) + \sum_{s_{t+2}} P(s_{t+2} \mid s_{t+1}, a_{t+1}) \left[r(s_{t+2}) + \cdots \right] \right]$$

1. **Exponential tree of futures**
   At each step, you must sum over all possible next states and actions — the tree branches exponentially
2. **Requires a model of the world**
   You need the transition probabilities P(s'|s,a) and reward function r(s) — the full model of the environment
3. **Model may be unknown**

   In most real-world settings, the agent does not have access to the transition model

4. **Even with a model, planning is expensive**
   Computing the exact value requires searching through the full tree — intractable for large state spaces

Two approaches to RL

How do we estimate value without exhaustive computation?

Model-based

- Learn a model: P(s'|s,a), r(s)
- Plan by simulating trajectories
- Flexible — adapts instantly to changes
- Computationally expensive
- Brain: hippocampus, PFC
- → Next lecture

Model-free

- Learn value directly from experience
- No model needed — just samples
- Fast at decision time (cached values)
- Slow to adapt (habitual)
- Brain: basal ganglia, dopamine
- → **This lecture**

Temporal difference (TD) learning

Key idea: use the Bellman equation as an error signal

TD error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

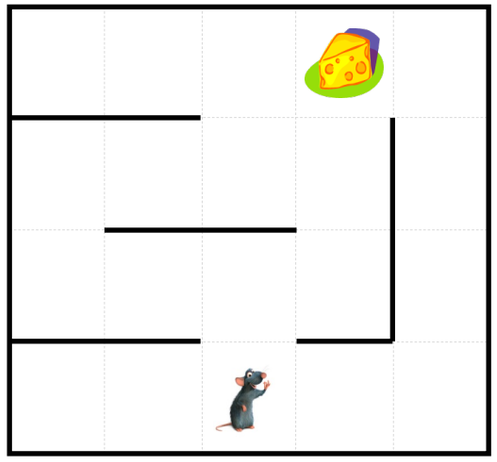
actual predicted

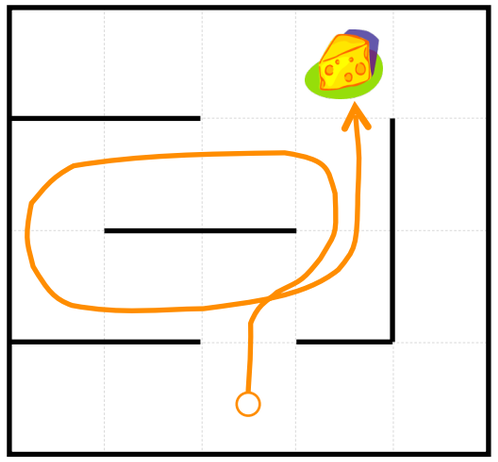
Update rule:

$$V(s_t) \leftarrow V(s_t) + \alpha \cdot \delta_t$$

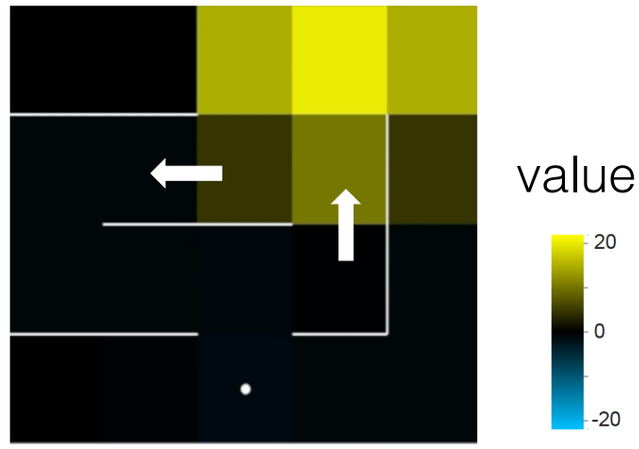
No model needed — learn from single transitions (s, a, r, s')

Temporal difference (TD) learning
Solving credit assignment by moment-by-moment error signals





Temporal difference (TD) learning



- Reward comes only after a sequence of actions.
- Which actions were responsible for obtaining reward?
- Temporal credit assignment

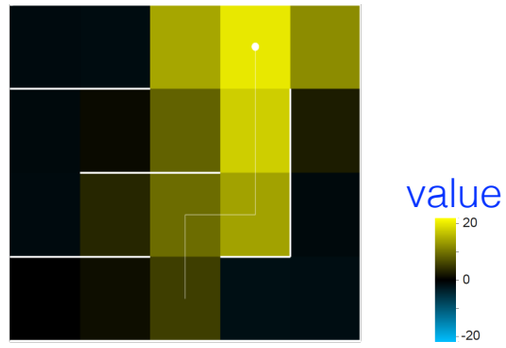
- Learn values of states and actions
- Compute TD error at each step
- Update value

Reinforcement learning (RL) methods

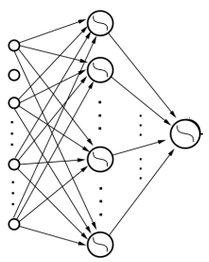
- Tabular method

Direct mapping
(aka look-up table)

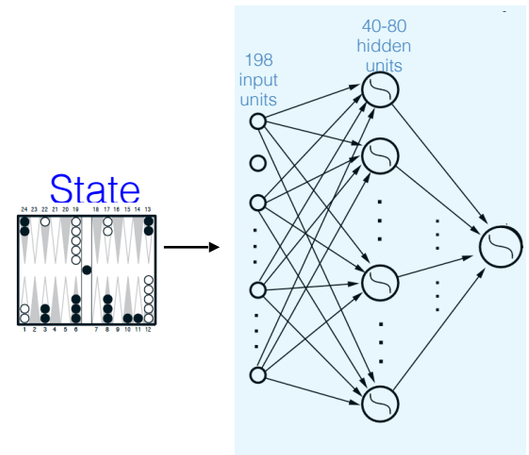
State ⟶ Value or Policy



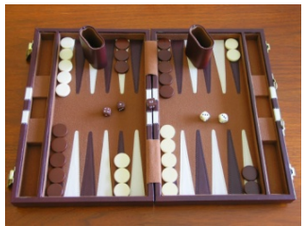
- Function approximation

State ⟶  ⟶ Value or Policy

e.g. Neural network
= function approximator



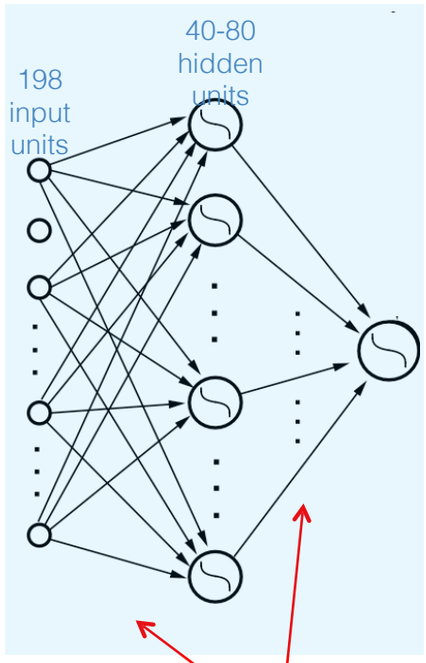
Synaptic weights indirectly determine the mapping between state and value (policy).



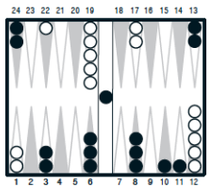
TD-Gammon:
Multi-layer neural network with TD error

$$Value = f(State)$$

Neural network
(function approximator)



State



Value
(probability of winning)

- To handle the large number of input patterns (states)
- Generalization

TD error $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$

Gradient descent
(adjust weights to minimize TD errors)

Q-learning

TD learning applied to the action-value function (Watkins, 1989):

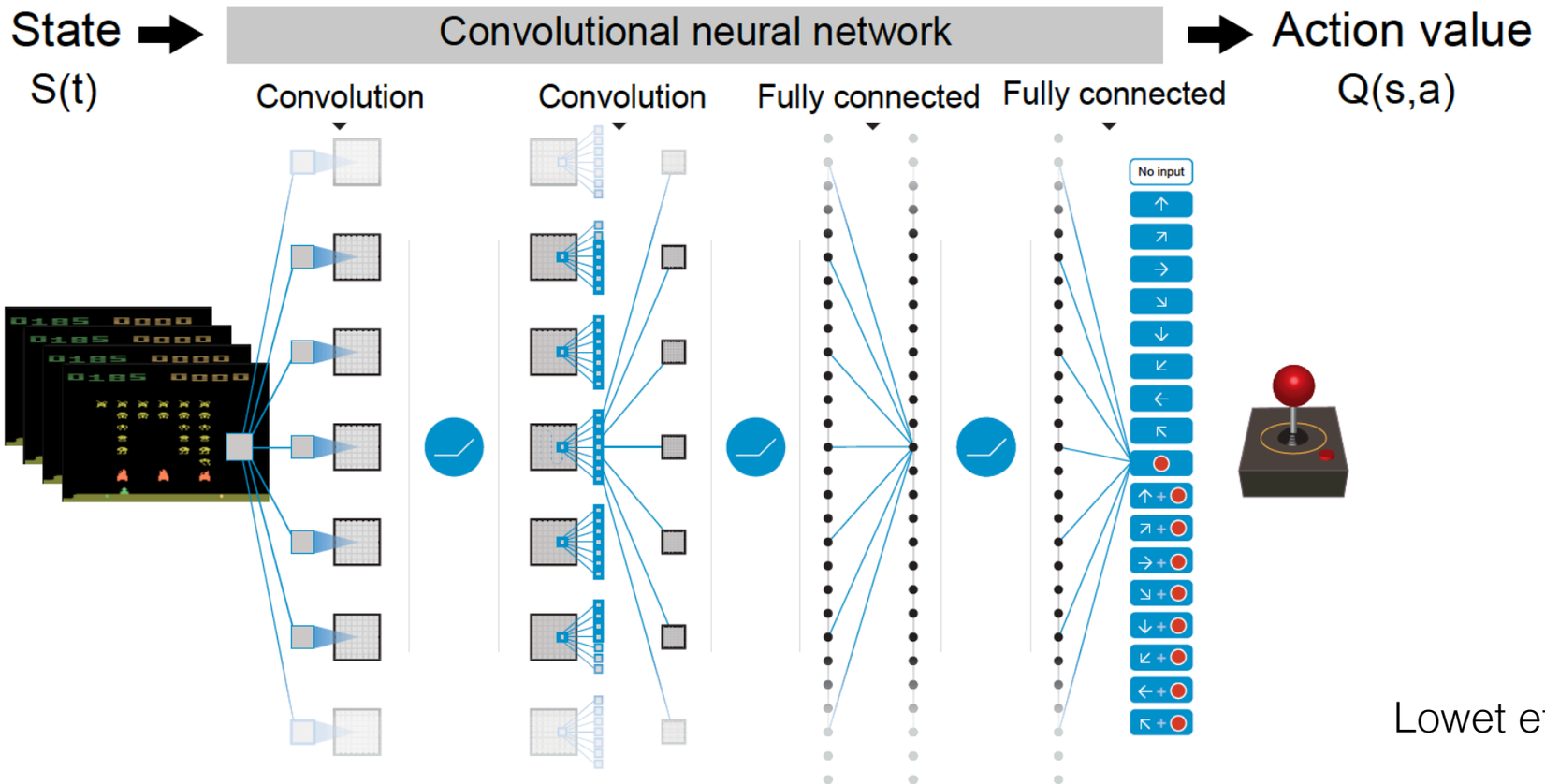
$$\delta_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \cdot \delta_t$$

Properties of Q-learning:

- Off-policy: can learn optimal Q* from any behavior
- Model-free: no knowledge of P(s'|s,a) required
- Convergence guaranteed (tabular) under mild conditions
- Foundation for Deep Q-Networks (Mnih et al., 2015)

Deep Q-Network (DQN) in Atari



Lowet et a. 2020

- Loss function:

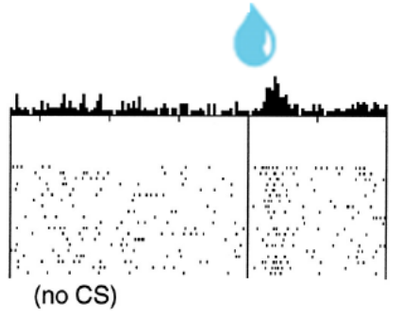
$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim \mathrm{U}(D)}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta_i^-) - Q(s,a;\theta_i)\right)^2\right]$$

θ: model parameters (e.g. weight)

⇨ Stochastic gradient descent to reduce *L*

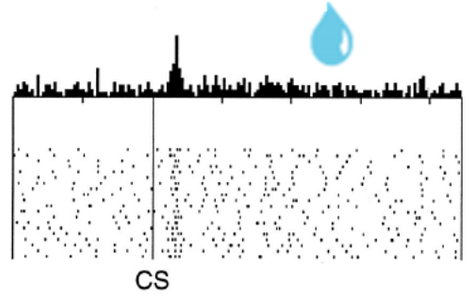
Dopamine neurons encode TD error

No prediction
Reward occurs

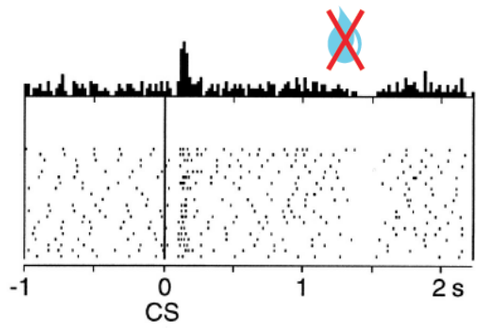


CS predicts reward
Reward occurs

(CS: conditioned stimulus)



CS predicts reward
No reward occurs



(Schultz, Montague, Dayan, 1997)

Dopamine neurons encode TD error

Schultz, Dayan & Montague (1997)

(1) No prediction, reward occurs

Unexpected reward → dopamine fires

`Burst (δ > 0)`

(2) Cue predicts reward, reward occurs

After learning, response shifts to the predictive cue (δ > 0 at cue, δ = 0 at reward)

`Burst at cue, nothing at reward`

(3) Cue predicts reward, no reward

Expected reward omitted → dopamine dips below baseline at expected time

`Dip (δ < 0)`

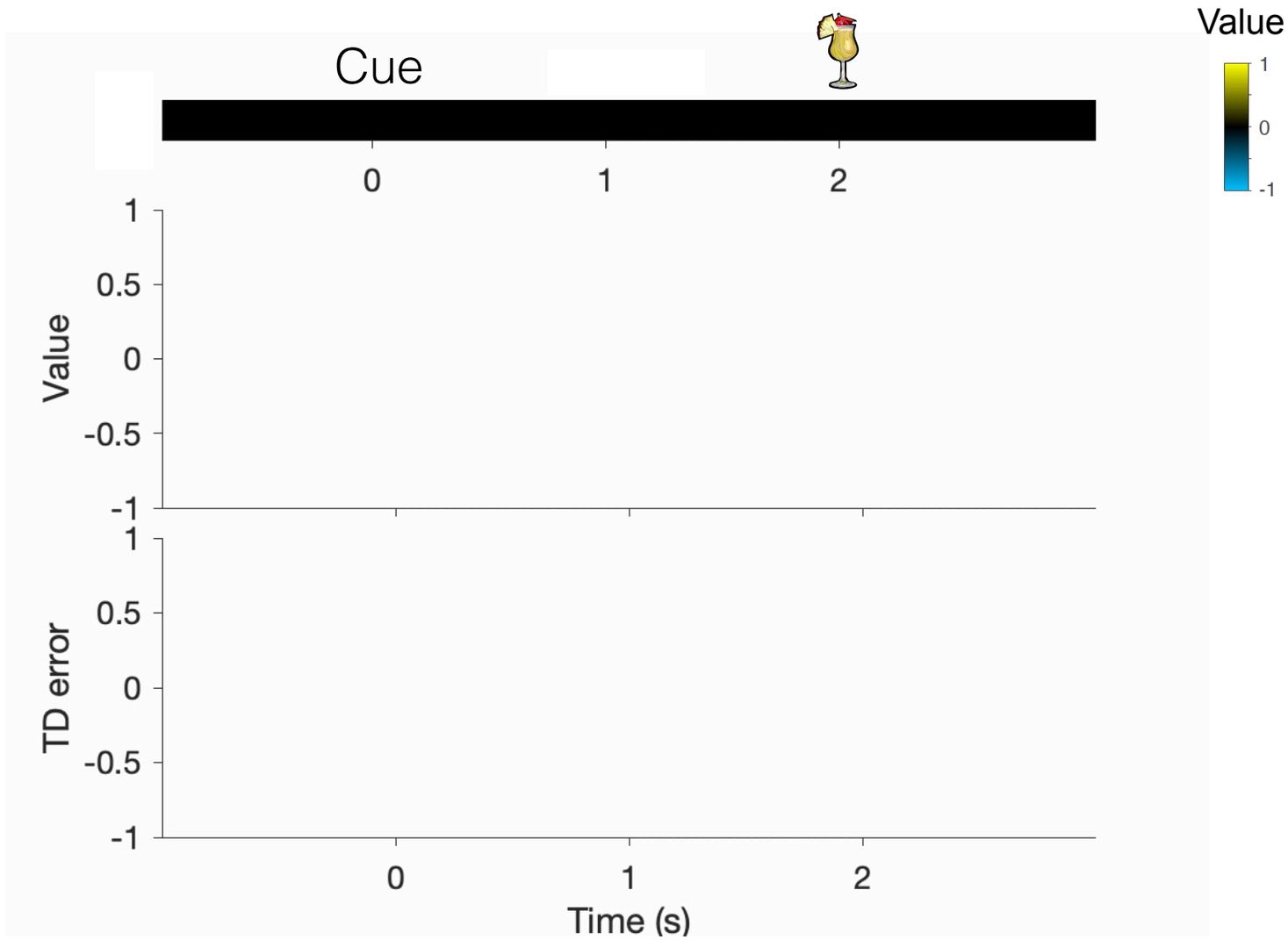
→ **Dopamine firing** $= \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$

Applying TD learning to classical conditioning

Tapped delay line
Complete serial compound

Montague et al., 1996; Schultz et al., 1997

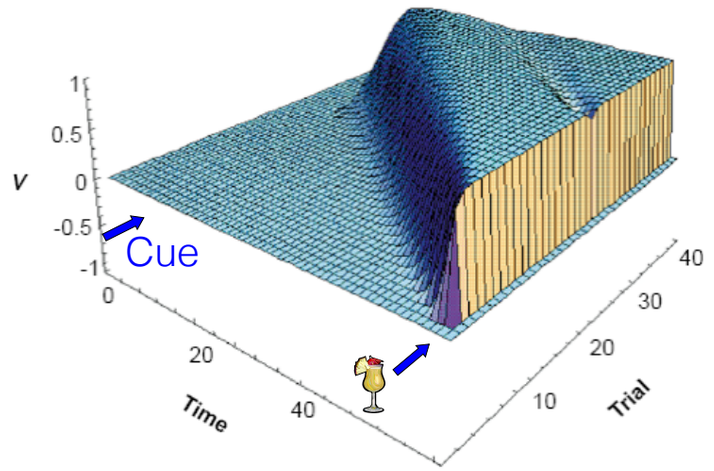
Applying TD learning to classical conditioning



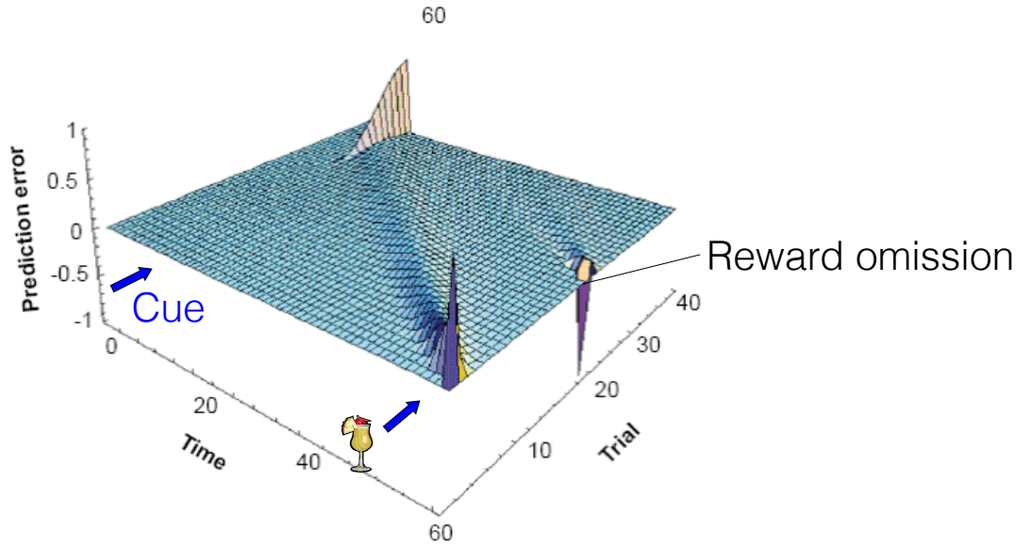
Montague et al., 1996; Schultz et al., 1997

Can we observe a temporal shift of dopamine signals during learning?

Value



TD error

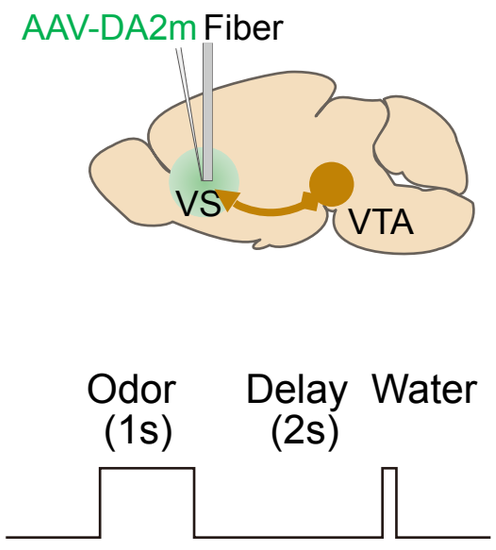


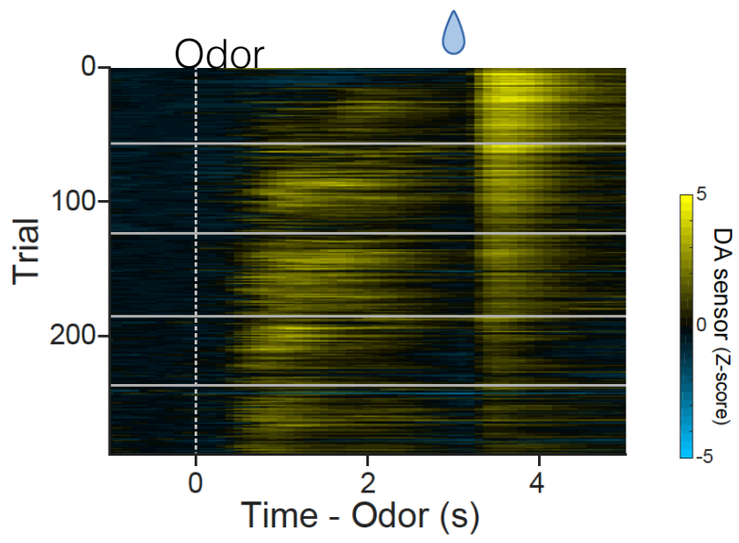
Montague et al., 1996; Schultz et al., 1997

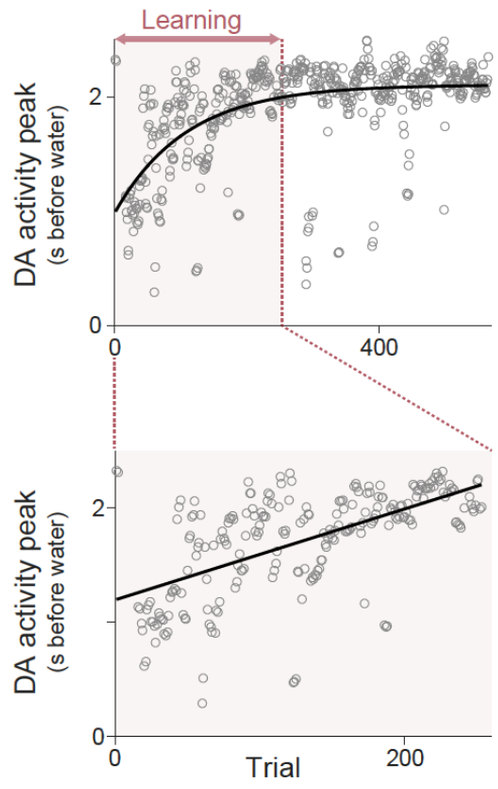
A temporal shift during learning

- First time learning (naïve mice)

Dopamine sensor

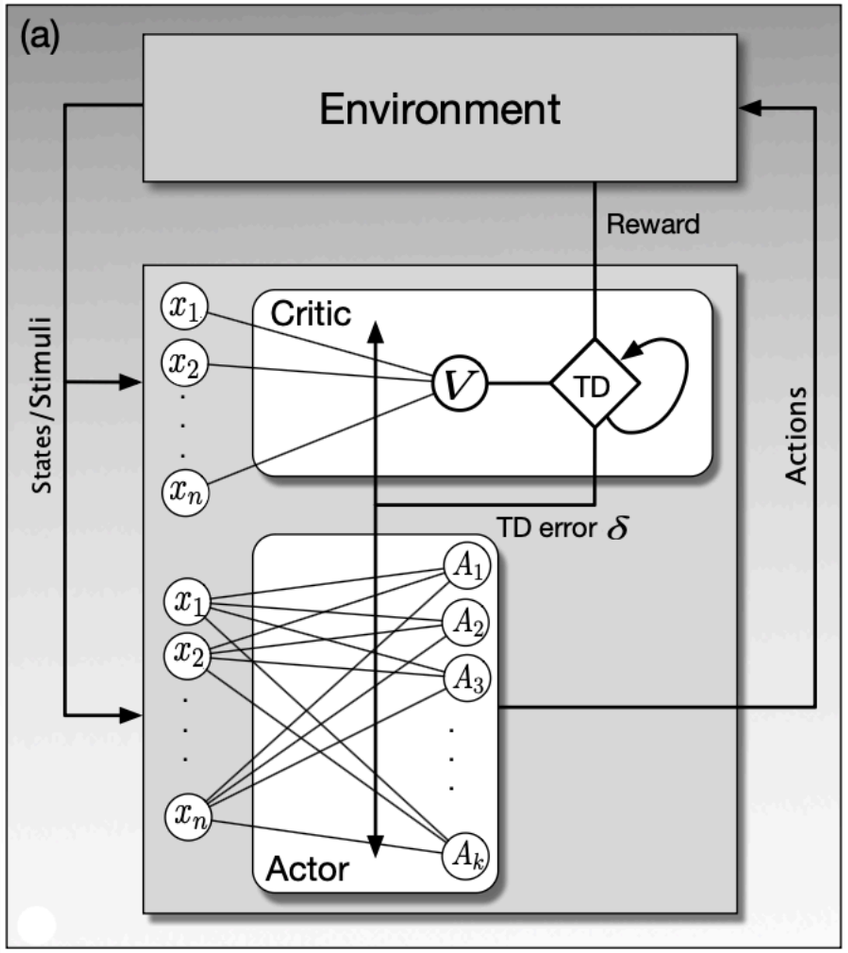


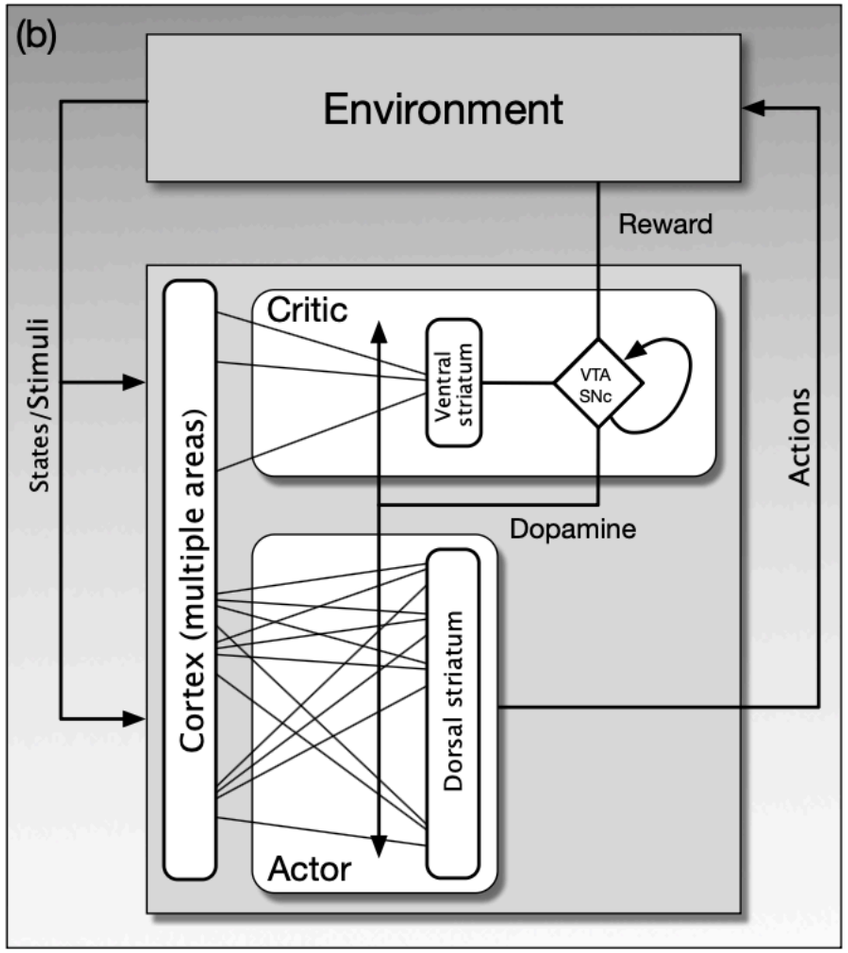




Amo, Matias, Yamanaka, Tanaka, Uchida, Watabe-Uchida, *Nat. Neurosci.*, 2022

Circuit implementation of TD learning





The basal ganglia as actor–critic

Critic (ventral striatum)

- Learns V(s) — expected future reward
- Computes $\delta = r + \gamma V(s') - V(s)$
- Sends δ to dopamine neurons
- $V(s) \leftarrow V(s) + \alpha \cdot \delta$

Actor (dorsal striatum)

- Learns $\pi(a|s)$ — which action to take
- D1 neurons (Go): strengthened by $\delta > 0$
- D2 neurons (NoGo): strengthened by $\delta < 0$
- Dopamine modulates D1/D2 differentially

Dopamine signal δ:

$\delta > 0$ (better than expected): D1 LTP, D2 LTD → reinforce action

$\delta < 0$ (worse than expected): D1 LTD, D2 LTP → suppress action

Three-factor learning rule

How does a synapse know it was responsible for the reward?

$$\Delta w_{ij} = \eta \cdot z_{ij}(t) \cdot \delta(t)$$

Factor 1: Hebbian activity f(pre, post)

Pre/post co-activity triggers a biochemical tag at the synapse

Factor 2: Eligibility trace $z_{ij}(t)$

The tag decays over ~1–2 seconds (Yagishita et al., 2014), marking the synapse as eligible

Factor 3: Neuromodulatory signal δ (dopamine)

Global reward prediction error — only tagged synapses are modified

The full Critic-Actor model

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1},\mathbf{w}) - \hat{v}(S_t,\mathbf{w}),$$

$$\mathbf{z}_t^{\mathbf{w}} = \lambda^{\mathbf{w}} \mathbf{z}_{t-1}^{\mathbf{w}} + \nabla \hat{v}(S_t,\mathbf{w}), \qquad \text{Critic}$$

$$\mathbf{z}_t^{\boldsymbol{\theta}} = \lambda^{\boldsymbol{\theta}} \mathbf{z}_{t-1}^{\boldsymbol{\theta}} + \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta}), \qquad \text{Actor}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta_t \mathbf{z}_t^{\mathbf{w}},$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \delta \mathbf{z}_t^{\boldsymbol{\theta}},$$

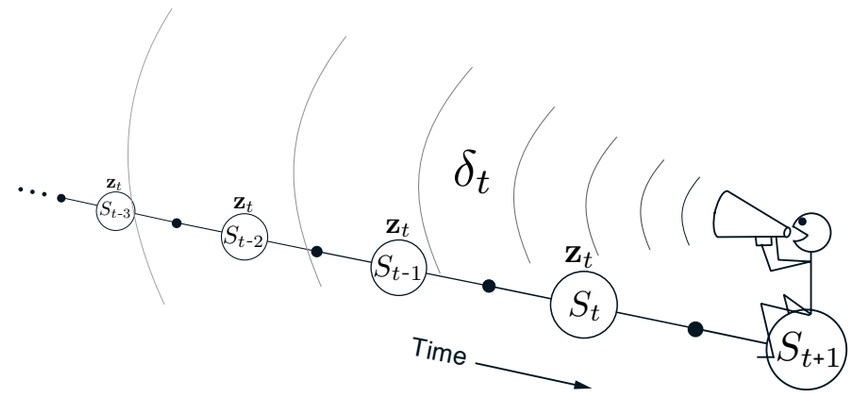


Figure 12.5: The backward or mechanistic view of TD(λ). Each update depends on the current TD error combined with the current eligibility traces of past events.

$$\hat{v}(s,\mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s).$$

$$\pi(1|s,\boldsymbol{\theta}) = 1 - \pi(0|s,\boldsymbol{\theta}) = \frac{1}{1+\exp(-\boldsymbol{\theta}^\top \mathbf{x}(s))}.$$

$$\nabla \ln \pi(A_t|S_t,\boldsymbol{\theta}) = \big(A_t - \pi(1|S_t,\boldsymbol{\theta})\big)\mathbf{x}(S_t).$$

$$\nabla \hat{v}(S_t, \mathbf{w}) = \mathbf{x}(S_t)$$

Summary: model-free RL in the brain

1. The Bellman equation defines value recursively: $V(s) = E[r + \gamma V(s')]$

2. Computing $Q(s,a)$ exactly requires a model and exponential search — motivates model-free methods

3. TD learning uses the Bellman error $\delta = r + \gamma V(s') - V(s)$ to update value from experience

4. Dopamine neurons in VTA/SNc fire exactly like δ — the reward prediction error

5. Basal ganglia implement actor–critic: ventral striatum (critic) + dorsal striatum (actor)

6. The three-factor rule (Hebbian tag × eligibility trace × dopamine) solves temporal credit assignment

7. Limitation: model-free RL is habitual — cannot plan or flexibly revalue → motivates model-based RL

→ **Next lecture: model-based RL, hippocampus, replay, and world models**